

ASSEMBLY ENGINE. AGENTS. FABRIC. PROOF.

A database engine in x86-64 assembly, the SaaS built around it, the Microsoft Fabric workload that decodes it, the game that proves it works, and the eleven agents that built it all.

ASMDB ENGINE | WWW.ASMDB.CLOUD | ASMDB.ANALYTICS | WWW.PIXELSLIME.CLOUD | PIXELSLIME ANALYTICS

THE STORY

Five things, one thread

Each one only exists because the one before it did. Nothing here is a mock-up.

01
asmdb

THE ENGINE

A transactional CRUD engine hand-written in x86-64 assembly. NASM -f bin, no linker, no libc. 43,741 bytes.

→

02
asmdb.cloud

THE SERVICE

One database, one Container App, one writer. Entra ID at the front, private endpoints everywhere else.

→

03
asmdb.analytics

THE WORKLOAD

A Microsoft Fabric workload that turns the engine's change log into Delta tables in a Lakehouse.

→

04
pixelslime.cloud

THE PROOF

One AI-generated slime a day, stored in 175 bytes and fingerprinted on Polygon. A real product on the real service.

→

05
PixelSlime Analytics

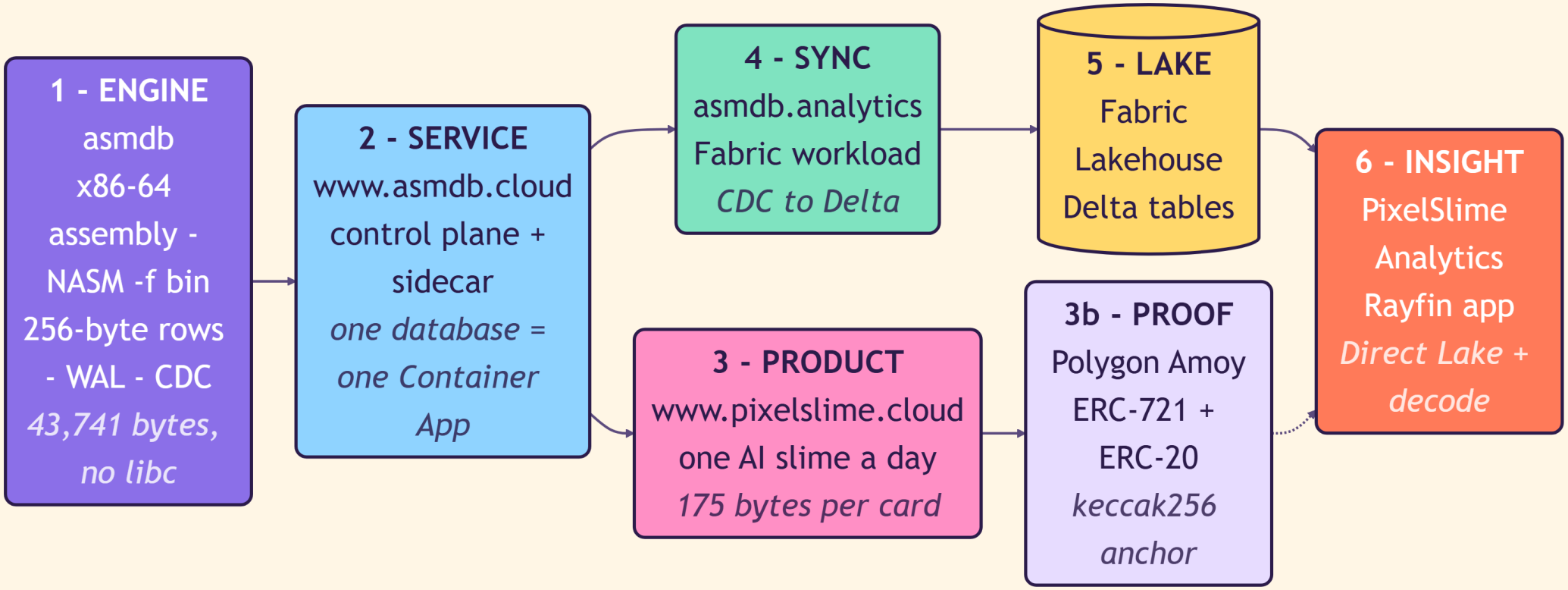
THE PAYOFF

A Fabric app that decodes those bytes back into KPIs over Direct Lake.

The thread. An engine is only interesting if someone runs it. A service is only interesting if something depends on it. A pipeline is only interesting if it moves real bytes. So each layer exists to put the one below it under load — and the last one reads the first one's rows back out.

One byte's journey, end to end

A row written by 43 kilobytes of assembly ends up as a measure in a semantic model. This is the only slide you have to remember.



AGENDA

How the next ninety minutes go

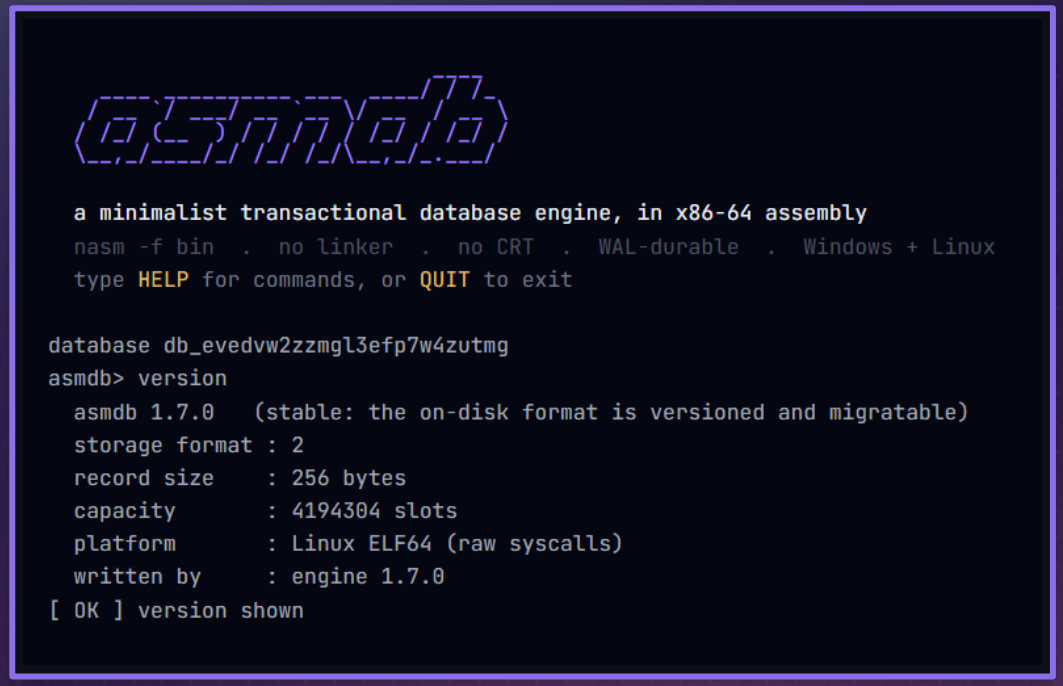
01	**The engine**	Why 256 bytes, why no SQL, and why the store is the index	20 MIN
02	**The service**	Turning a 43 KB binary into a multi-tenant SaaS on Azure	15 MIN
03	**The workload**	Change data capture from an assembly engine into Fabric Delta tables	15 MIN
04	**The product**	PixelSlime: 175 bytes, one codec, four smart contracts	20 MIN
05	**The payoff**	Decoding the bytes back into KPIs with a Rayfin Fabric app	10 MIN
06	**The method**	How eleven agents built this without stepping on each other	10 MIN

01

THE ENGINE

asmdb

A transactional database engine, hand-written in x86-64 assembly.
No linker. No C runtime. Nothing underneath it.



DEFINITION

Not a wrapper. Not a binding. The engine itself.

One source file assembles straight to an executable. There is no runtime between the SQL-less command parser and the syscall.

Written in assembly

x86-64, assembled with NASM in -f bin mode. The PE and ELF headers are emitted by hand, the import table is hand-built, and Linux talks in raw syscalls.

Fixed-shape rows

Every record is exactly 256 bytes. The row array is the index: open addressing with linear probing over a Fibonacci hash, 0x9E3779B97F4A7C15.

Durable by construction

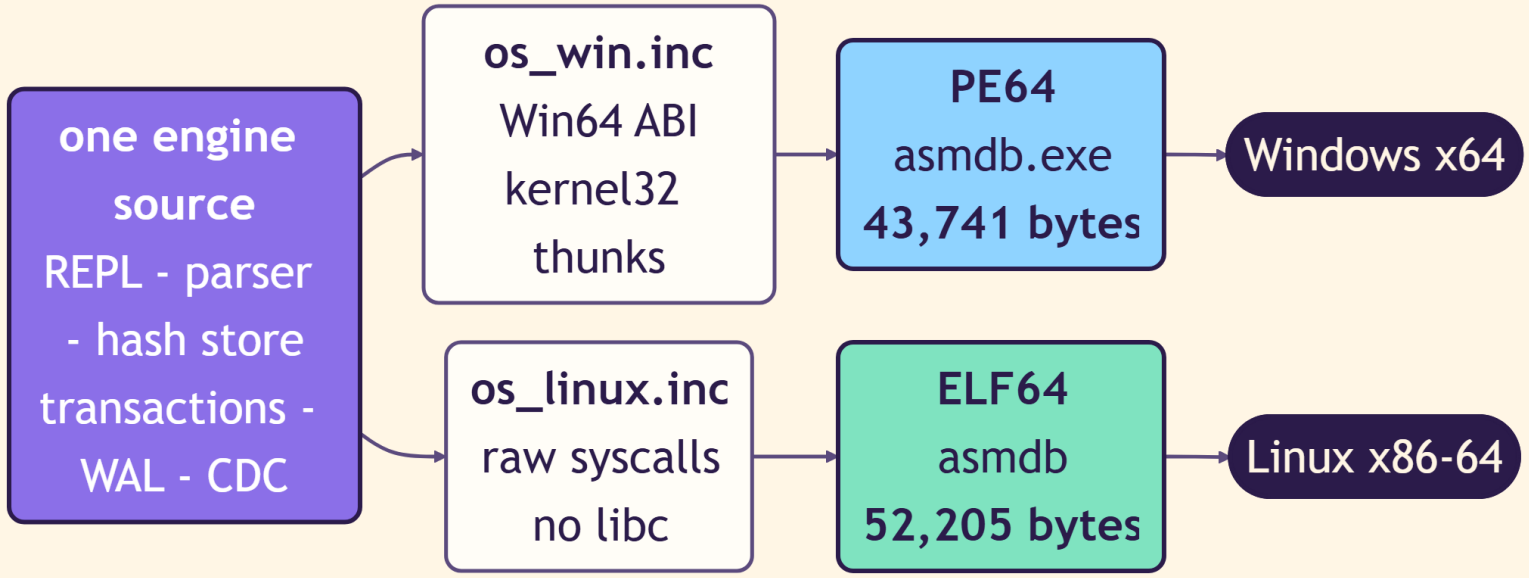
A write-ahead log with a CRC-32 commit marker, a two-fsync commit point, and an append-only change log written before the data file is touched.

One writer, many readers

A byte-range lock on Windows, flock on Linux. Unlimited --reader sessions. No planner, no joins, no secondary indexes, and none of the code that would need them.

One source, two operating systems, two binaries

The only thing that differs between Windows and Linux is how the engine asks the kernel for a file.



ASMDB.EXE · PE64

43,741 B

Windows x64. Smaller than most JPEGs on this slide.

ASMDB · ELF64

52,205 B

Linux x86-64, raw syscalls, no libc at all.

THE RECORD

256 bytes, and never one more

A power of two, so the address of slot N is a shift instead of a multiply. Every design decision downstream falls out of this one.



USABLE CONTENT

175 bytes

USABLE TAG

39 bytes

STATUS = 2

tombstone

ADDRESSING

shl rax, 8

Why it matters. Because 256 = 2^8, finding a record is `address = table_base + (slot × 256)` — two instructions. No page table, no extent manager, no buffer pool, no catalog lookup. There is nothing to look up because there is nothing to look up *in*.

CAPACITY

The store is the index — so the size is decided up front

asmdb computes the slot from the id, then walks forward. That only works if it knows how many slots exist before the first insert.

CAPACITY	SLOTS	REGION	USABLE ROWS (75%)
Small	524,288	128 MiB	393,216
Medium	2,097,152	512 MiB	1,572,864
Large	4,194,304	1 GiB	3,145,728

`4,194,304 slots × 256 bytes = 1,073,741,824 bytes = exactly 1 GiB`

Why not just grow it?

- The slot for an id depends on the total number of slots, so resizing rehashes every single record.
- You cannot append empty slots to the file and call it a day.
- A crash-safe resize needs migration state in the header, two regions, a write pause, reader coordination and CDC validation.
- **Dynamic resizing is an explicit, documented non-goal.**

THE 75% RULE

Why a quarter of the table is deliberately left empty

Linear probing is fast until the table fills up. Then clusters form, and the cost of an unsuccessful search stops being a constant.

FILL RATE 50 %

2.5

slots examined

FILL RATE 75 %

8.5

slots examined

FILL RATE 90 %

50.5

slots examined

FILL RATE 95 %

200.5

slots examined

$0.5 \times (1 + 1 / (1 - load)^2)$

theoretical cost of an unsuccessful search

4,194,304 slots × 0.75 = 3,145,728 usable records

The threshold is a trade between storage density, predictable performance, collision count, search time, and room for new inserts. At 95% a single insert can scan hundreds of slots; in the worst case store_locate walks the entire table. Leaving 25% empty is not waste — it is the performance guarantee.

PERFORMANCE

It is fast because of what is missing

Every one of these is a subsystem asmdb never had to write, debug, or pay for at runtime.

- ~~B-tree~~
- ~~Variable-size pages~~
- ~~Extent manager~~
- ~~Page allocation~~
- ~~Catalog~~
- ~~Index rebalancing~~
- ~~Pointers between pages~~
- ~~Buffer manager~~

store_locate

```
; finding record N is two instructions
shl  rax, 8          ; slot * 256
add  rax, table_base ; done

; the slot itself is a multiply-shift
imul rax, GOLDEN     ; 0x9E3779B97F4A7C15
mov  cl, [g_cap_shift]
shr  rax, cl         ; keep the top bits
```

Memory-mapped, copy-on-write

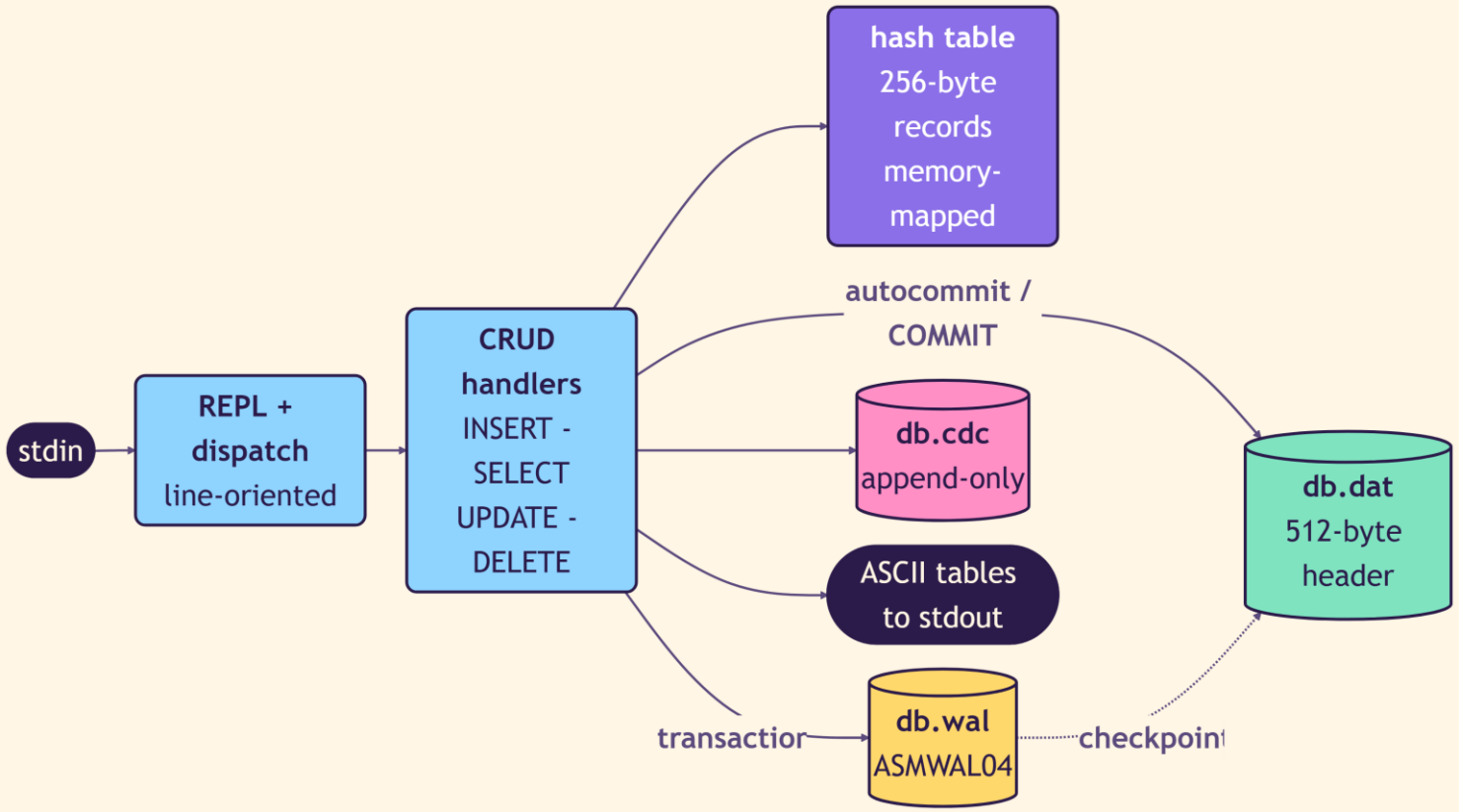
OPEN + COUNT
684 ms → 104 ms

PEAK WORKING SET
1,029 MB → 5 MB

ON A SPARSE FILESYSTEM
a 1 GiB region costs a few MiB until it is touched

One process, one table, one writer

stdin goes in, ASCII tables come out. The same code path serves the CLI, the sidecar, the REST API and the MCP server.



Three files, one database

`<db>.dat`
512-byte header, magic ASMDB, format version 2, capacity, live count, engine version, last_commit_seq

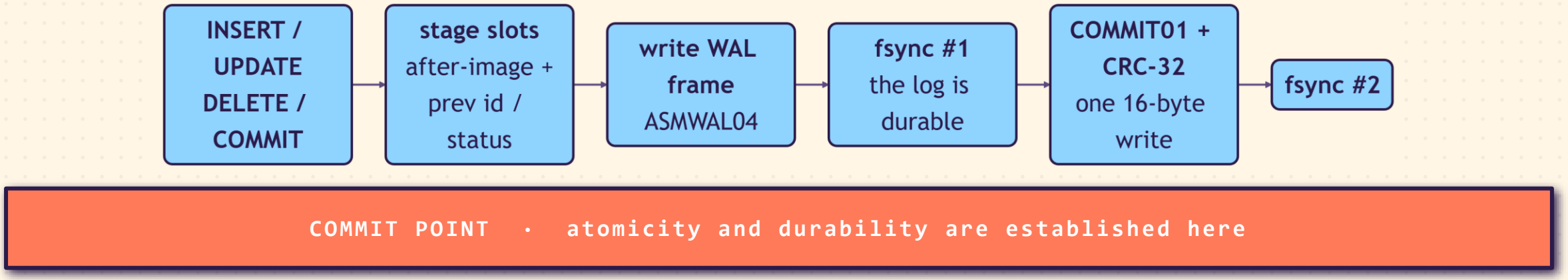
`<db>.wal`
ASMWAL04 staged frames, COMMIT01 marker, CRC-32, idempotent recovery

`<db>.cdc`
append-only, one frame per committed transaction, RESET for whole-table operations

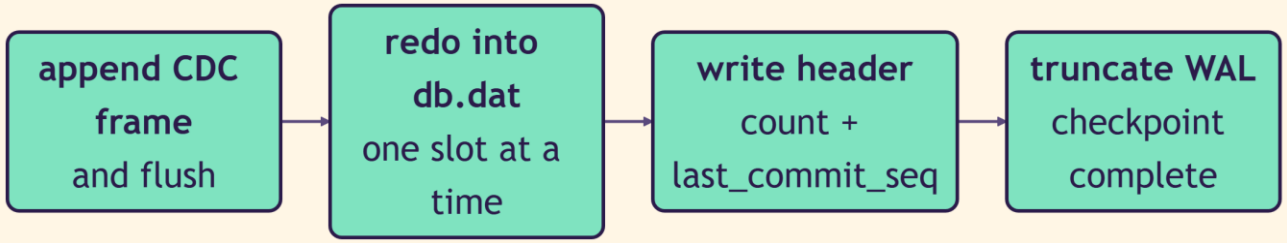
The commit point is a place, not a feeling

Two fsyncs and a 16-byte marker. Everything before it can be thrown away; everything after it is pure, replayable redo.

BEFORE — nothing is visible yet

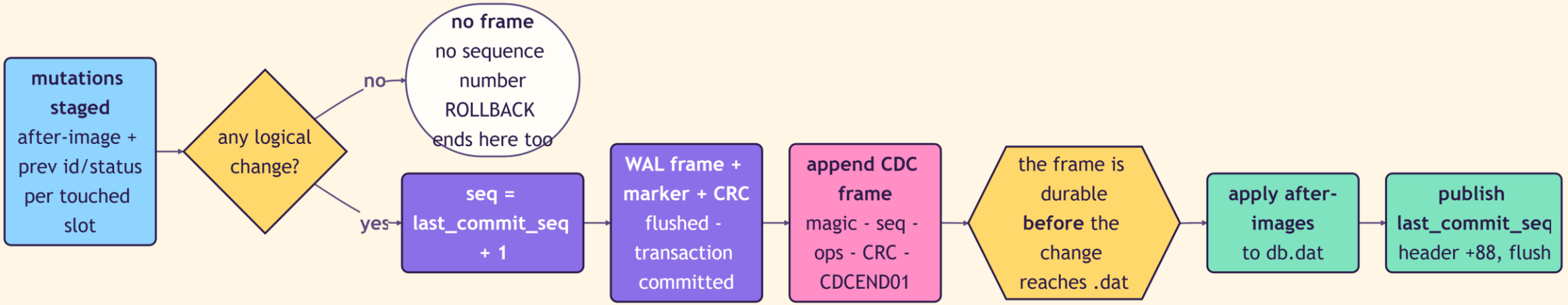


AFTER — pure redo, safe to replay



CDC is not a feature bolted on afterwards

The change frame is flushed before the change reaches the data file. If the log says it happened, it happened.



Ordered

One frame per committed transaction, numbered by a monotonic last_commit_seq published at header offset +88.

Honest

A rolled-back or no-op transaction produces no frame and consumes no sequence number. Nothing is invented.

Auditable

Gaps and truncation are detectable states, not surprises. TRUNCATE emits RESET so a consumer knows to rebuild.

THE SURFACE

No SQL. Twenty-six commands.

There is no parser to exploit, no planner to mispredict, and no query that can accidentally cost a million rows.

CRUD

INSERT · SELECT · UPDATE · DELETE · TRUNCATE

READ

FIND · RANGE · COUNT

TRANSACTIONS

BEGIN · COMMIT · ROLLBACK

CATALOG

TABLES · DATABASES · SCHEMA · TYPES · VERSION

OPERATIONS

BACKUP · RESTORE · VERIFY · CDCTRIM · BENCH

DIAGNOSTICS

FORMAT · PAGE · HELP · EXIT

Also: --reader for unlimited read-only sessions, and --upgrade for a versioned, migratable on-disk format.

BENCHMARKS

What 43 kilobytes buys you

Measured on the same machine, same workload, against SQLite in its comparable mode.

IN-RAM INSERT

12.38 M

rows per second
12.3× SQLite

DURABLE BULK LOAD

1.17 M

rows per second
1.4× SQLite

DURABLE, ROW BY ROW

932

rows per second
two fsyncs each

STDIO TRANSACTIONS

5,932

rows per second
9.2× SQLite

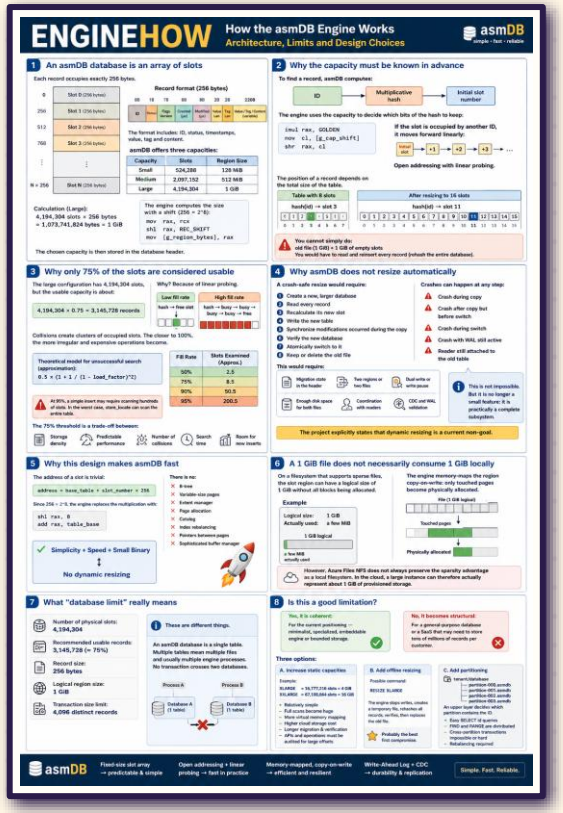
The honest asterisk

asmdb is faster because it does less. There is no query planner to warm up, no page cache to manage, no B-tree to descend and no schema to consult. The numbers above are real, and they are also the entire argument for the design: if you accept a fixed-size row and a single table, you can delete most of a database engine and keep the part that matters — a durable, ordered, crash-safe write path.

REFERENCE

The engine on one page

Slot array, open addressing, the 75% rule, why there is no automatic resize, and what "database limit" actually means.



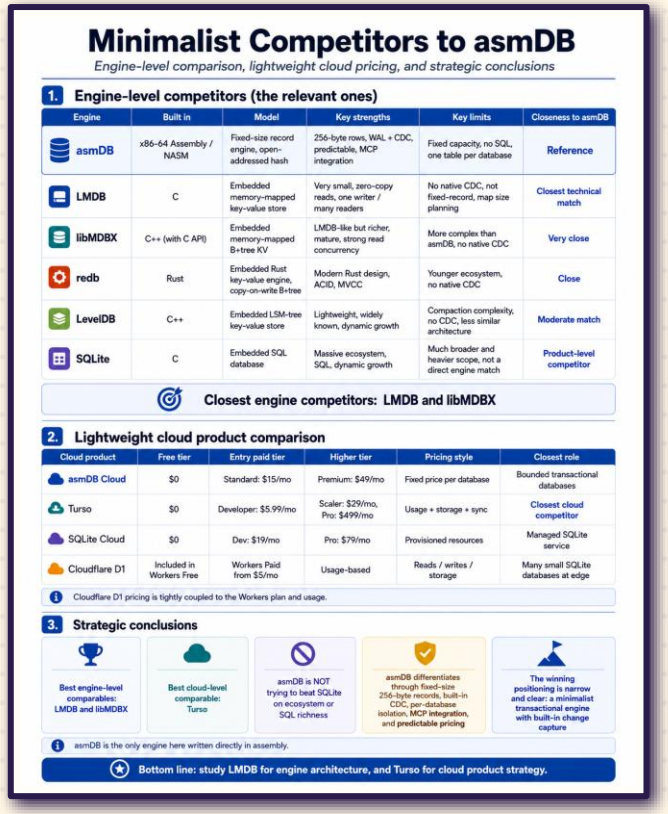
REFERENCE SHEET · shared as a full-size handout

1. **An asmdb database is an array of slots**
   Each record occupies exactly 256 bytes, and the slot number is derived from the id.
2. **The capacity must be known in advance**
   The engine uses the capacity to decide how many bits of the hash to keep.
3. **Only 75% of the slots are usable**
   Because clusters form under linear probing, and cost explodes past that point.
4. **It does not resize automatically**
   A crash-safe resize is a complete subsystem, not a small feature. It is a non-goal.
5. **A 1 GiB file need not cost 1 GiB**
   Sparse files and copy-on-write mapping mean only touched pages are allocated.
6. **Transaction limit: 4,096 records**
   One database is one table. No transaction crosses two databases.

POSITIONING

Who asmdb is actually competing with

Not SQLite. The honest comparables are embedded, memory-mapped key-value engines.



Minimalist Competitors to asmDB

Engine-level comparison, lightweight cloud pricing, and strategic conclusions

1. Engine-level competitors (the relevant ones)

Engine	Built in	Model	Key strengths	Key limits	Closeness to asmDB
asmDB	x86-64 Assembly / NASM	Fixed-size record engine, open-addressed hash	256-byte rows, WAL + CDC, predictable, MCP integration	Fixed capacity, no SQL, one table per database	Reference
LMDB	C	Embedded memory-mapped key-value store	Very small, zero-copy reads, one writer / many readers	No native CDC, not fixed-record, map size planning	Closest technical match
libMDBX	C++ (with C API)	Embedded memory-mapped B+tree KV	LMDB-like but richer, mature, strong read concurrency	More complex than asmDB, no native CDC	Very close
redb	Rust	Embedded Rust key-value engine, copy-on-write B+tree	Modern Rust design, ACID, MVCC	Younger ecosystem, no native CDC	Close
LevelDB	C++	Embedded LSM-tree key-value store	Lightweight, widely known, dynamic growth	Compaction complexity, no CDC, less similar architecture	Moderate match
SQLite	C	Embedded SQL database	Massive ecosystem, SQL, dynamic growth	Much broader and heavier scope, not a direct engine match	Product-level competitor

Closest engine competitors: LMDB and libMDBX

2. Lightweight cloud product comparison

Cloud product	Free tier	Entry paid tier	Higher tier	Pricing style	Closest role
asmDB Cloud	$0	Standard: $15/mo	Premium: $49/mo	Fixed price per database	Bounded transactional databases
Turso	$0	Developer: $5.99/mo	Scaler: $29/mo, Pro: $499/mo	Usage + storage + sync	Closest cloud competitor
SQLite Cloud	$0	Dev: $19/mo	Pro: $79/mo	Provisioned resources	Managed SQLite service
Cloudflare D1	Included in Workers Free	Workers Paid from $5/mo	Usage-based	Reads / writes / storage	Many small SQLite databases at edge

Cloudflare D1 pricing is tightly coupled to the Workers plan and usage.

3. Strategic conclusions

- Best engine-level comparables: LMDB and libMDBX
- Best cloud-level comparable: Turso
- asmDB is NOT trying to beat SQLite on ecosystem or SQL richness
- asmDB differentiates through fixed-size 256-byte records, built-in CDC, per-database isolation, MCP integration, and predictable pricing
- The winning positioning is narrow and clear: a minimalist transactional engine with built-in change capture

asmDB is the only engine here written directly in assembly.

Bottom line: study LMDB for engine architecture, and Turso for cloud product strategy.

REFERENCE SHEET · full comparison in the handout

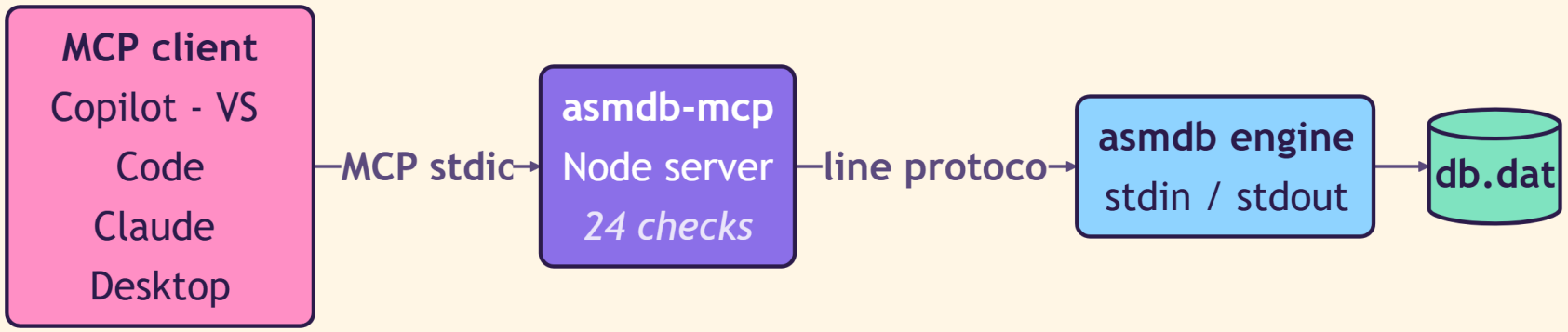
LMDB	C	Closest technical match
libMDBX	C++	Very close, richer, more complex
redb	Rust	Close: ACID, MVCC, no native CDC
LevelDB	C++	Moderate: LSM-tree, no CDC
SQLite	C	A product-level competitor, not an engine match

asmdb differentiates on fixed-size 256-byte records, built-in change capture, per-database isolation, MCP integration and predictable pricing. It is the only engine in this table written directly in assembly.

AGENTS

The engine is also a tool an agent can hold

An MCP server wraps the same stdio protocol, so a coding agent can create, query and verify a real database without a driver.



Same protocol

The MCP server speaks the engine's line protocol over stdin and stdout. There is no second implementation to keep in sync.

24 checks

The server ships with its own test suite covering every tool it exposes, run on both Windows and Linux.

Real state

The agent operates on a real .dat file with a real WAL. Nothing is simulated, so nothing lies about durability.

HONESTY

What asmdb is not

A design is only credible if it can name the things it refuses to do.

No SQL

No joins, no planner, no secondary indexes, no CREATE TABLE.

One table per database

Multiple tables means multiple files and usually multiple processes.

No transaction spans two databases

Cross-database atomicity does not exist and is not simulated.

Fixed capacity

Chosen at creation. Growing means creating a new database and migrating.

4,096 records per transaction

A hard limit, enforced, not a guideline.

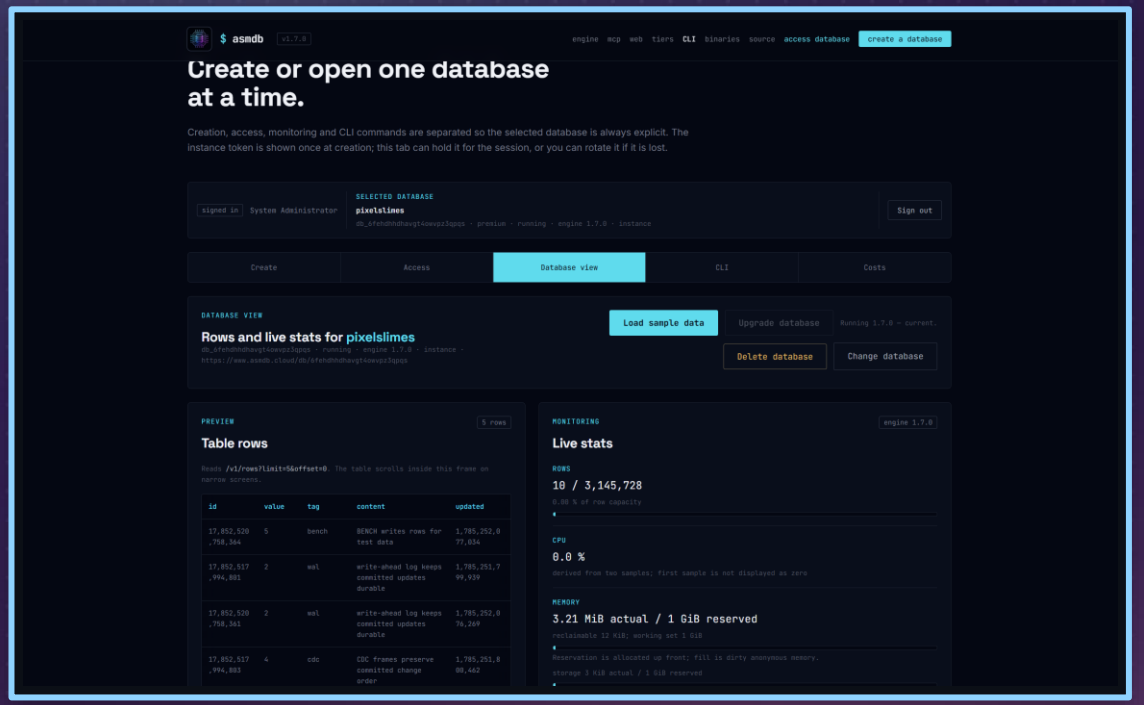
No read replicas or failover

Not built. The change log is the replication primitive, not the replication system.

02

THE SERVICE

www.asmdb.cloud



A 43-kilobyte binary is not a product. This is what it took to make one: a control plane, a sidecar, and one Container App per database.

THE GAP

Everything the engine deliberately does not do

The engine is single-writer, single-table and file-based. A service has to add the rest without changing any of that.

The engine gives you

- A process that owns one file
- A line protocol on stdin and stdout
- A byte-range lock for the single writer
- An append-only change log on disk
- A CRC-checked, crash-safe commit

The service must add

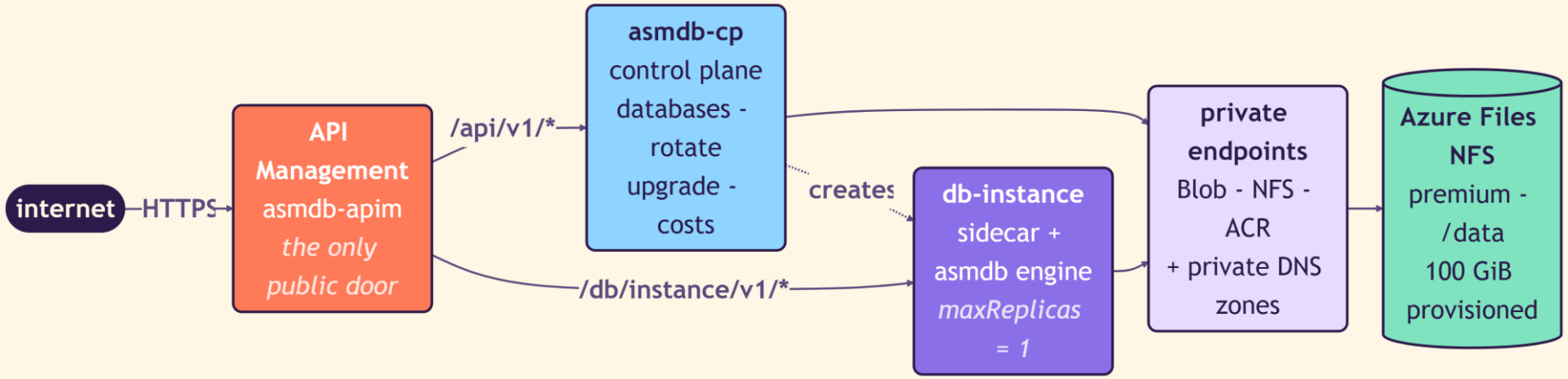
- Identity, tenancy and a token you can rotate
- An HTTP surface with a stable contract
- Isolation — one noisy tenant must not touch another
- A private, read-only way to publish that log
- Provisioning, monitoring, upgrade and a price

The rule that shaped every decision: the browser must never hold the instance token.

TOPOLOGY

One public door, and nothing else

API Management is the only thing on the internet. The control plane, the instances and the storage all live inside the VNet.



APIM

Developer tier. Routes /api/v1/* to the control plane and /db/<instance>/v1/* to the instance. A 60-second forward timeout, so a cold start can finish.

Control plane

Creates databases, rotates tokens, runs upgrades, reports costs. Internal Container Apps environment — not reachable from the internet.

Instance

The sidecar plus the engine, with maxReplicas pinned to 1 forever, because the engine allows exactly one writer.

Storage

Premium Azure Files NFS over a private endpoint, 100 GiB provisioned. Blob and ACR are private too, with their own DNS zones.

TENANCY

One database is one Container App

Not a schema. Not a row-level filter. A separate compute unit with its own mount subpath and its own token.

REPLICAS PER INSTANCE

1

A hard ceiling. Two replicas would mean two writers, and the engine's lock would simply reject the second.

TABLES PER DATABASE

1

One file, one table. Multi-table means multiple databases and therefore multiple instances.

SHARED STATE

0

No shared buffer pool, no shared catalog, no shared process. A tenant cannot see or slow another.

TOKENS STORED

0

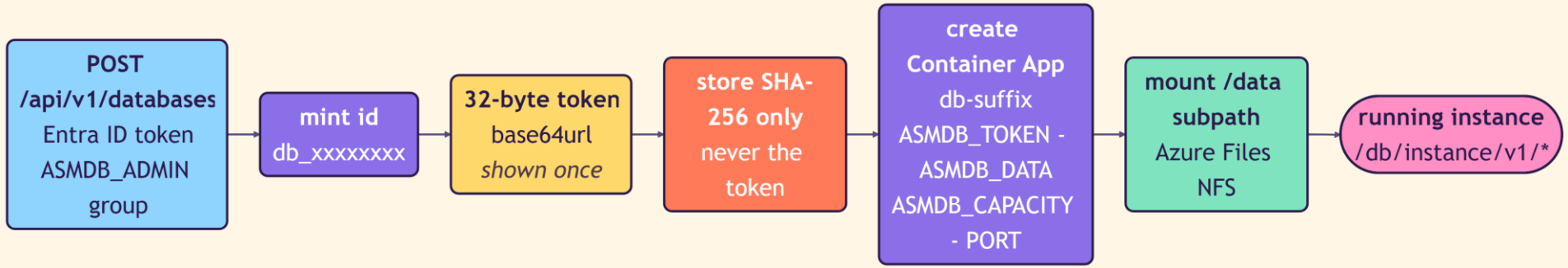
Only the SHA-256 hash is persisted. The token itself is displayed exactly once, at creation.

The cost of that choice

Per-tenant containers are more expensive than a shared cluster, and they are slower to start. In exchange you get an isolation story you can explain in one sentence, a blast radius of exactly one customer, and an upgrade path that is a container image tag. For a bounded, embeddable engine that trade is the right way round — for a database that must hold tens of millions of rows per customer, it would not be.

From POST to running instance

The token is generated, hashed, and forgotten. If the customer loses it, it is rotated — never recovered.



control plane

```
POST /api/v1/databases
Authorization: Bearer <Entra ID v2 access token>

201 Created
{ "id": "db_6fehdhhdhavgt4owvpz3qpqs",
  "token": "…shown exactly once…",
  "tier": "premium", "engine": "1.7.0" }
```

ADMIN AUTH
Entra ID v2 access token, membership of the ASMDB_ADMIN group checked on every management call.

BROWSER AUTH
PKCE. There is no client secret in the console, because there is nowhere safe to put one.

ROTATION
Two-phase. The pending token is sealed with AES-GCM until the client commits it, so a rotation cannot lock anyone out.

Two planes, one gateway

Management calls are authenticated with a human or an app identity. Data calls are authenticated with the instance token, and never leave the browser.

MANAGEMENT PLANE · Entra ID

GET	**/api/v1/version**	engine and platform versions
GET	**/api/v1/databases**	list the databases you own
POST	**/api/v1/databases**	provision a new instance
POST	**/api/v1/databases/{id}/wake**	explicit cold start
POST	**/api/v1/databases/{id}/rotate-token**	phase one of rotation
POST	**/api/v1/databases/{id}/upgrade**	move to a new engine build
GET	**/api/v1/costs**	what this account is actually costing

DATA PLANE · instance token

GET	**/db/{inst}/v1/rows**	paged rows, limit and offset
GET	**/db/{inst}/v1/count**	live row count
GET	**/db/{inst}/v1/find**	point lookup by id
GET	**/db/{inst}/v1/range**	ordered scan between two ids
POST	**/db/{inst}/v1/exec**	raw engine command
GET	**/db/{inst}/v1/stats**	rows, CPU, memory, storage
ANY	**/db/{inst}/v1/mcp**	the engine as an agent tool

Scale to zero, and be honest about waking up

Free and standard instances stop costing money when nobody is using them. Premium stays warm because analytics cannot wait.

FREE

$0 / mo

vCPU / memory
0.25 vCPU · 0.5 GiB

Idle behaviour
scales to zero

CDC to Fabric
not available

STANDARD

$15 / mo

vCPU / memory
0.5 vCPU · 1 GiB

Idle behaviour
scales to zero

CDC to Fabric
not available

PREMIUM

$49 / mo

vCPU / memory
1 vCPU · 2 GiB

Idle behaviour
stays warm

CDC to Fabric
included

Cold start

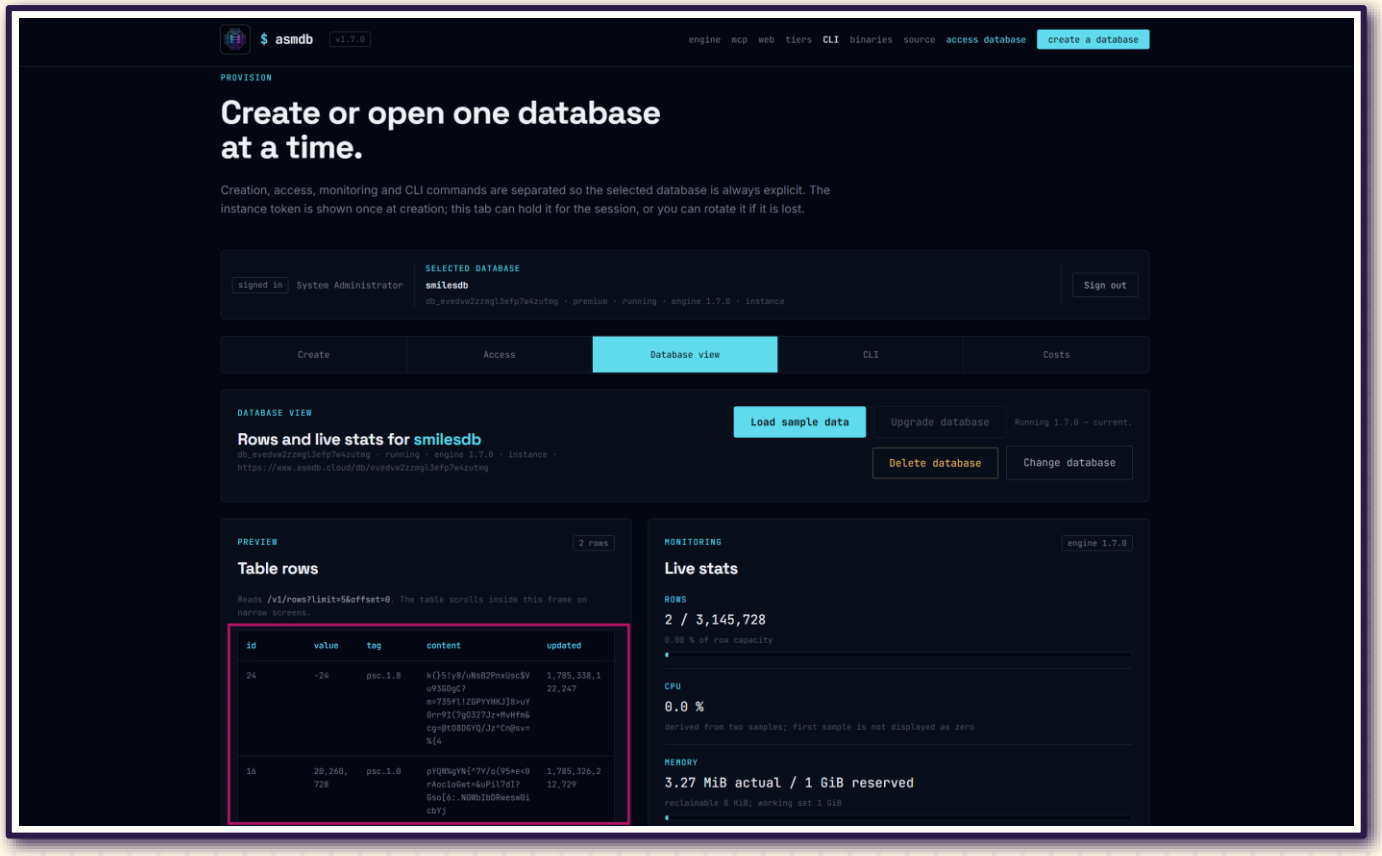
A scaled-to-zero instance has to be woken. Rather than hide that behind a silent retry, there is an explicit wake endpoint and a 60-second gateway timeout — so the first call after an idle period is slow and honest, rather than fast and wrong.

The unit economics nobody puts on a pricing page

The platform costs roughly **$141.76 per month** before a single customer exists — API Management, the Container Apps environment, a premium registry, the NFS share and Log Analytics. A free instance then adds about **$1.03 per month**. That fixed floor, not the per-instance cost, is what a fixed price per database has to amortise.

Create, access, monitor and pay — kept separate

www.asmdb.cloud · the selected database is always explicit, so no command can run against the wrong instance.



CREATE
Choose a capacity and a tier. The token appears once.

ACCESS
Hold the instance token for the session, or rotate it.

DATABASE VIEW
Live rows straight from /v1/rows?limit=5&offset=0.

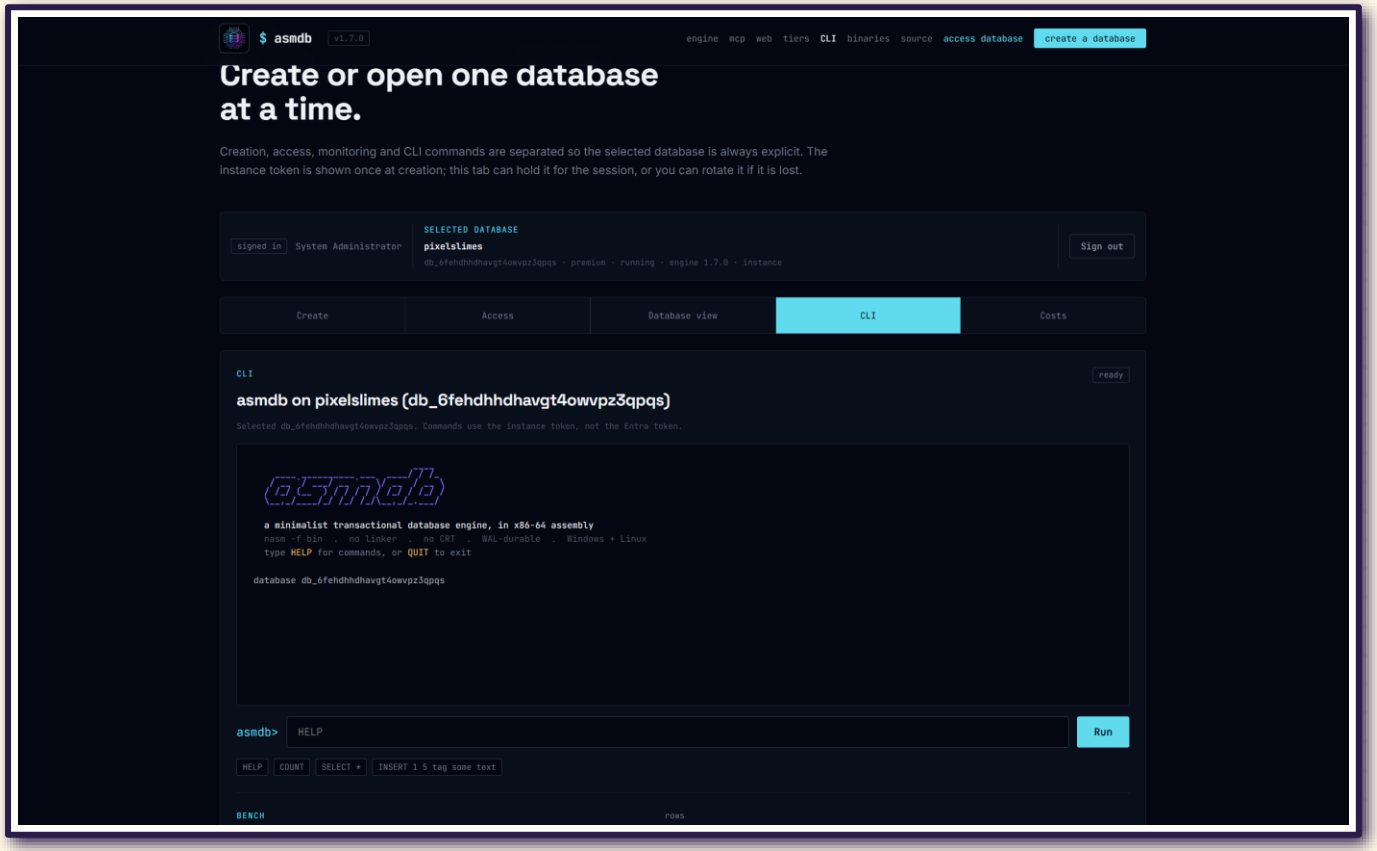
CLI
A browser terminal that speaks the engine's own protocol.

COSTS
What this database has actually consumed.

OBSERVABILITY

Live stats, straight from the engine

Not a metrics pipeline. The sidecar asks the running engine and reports what it says.



ROWS

10 / 3,145,728

0.00 % of row capacity used

MEMORY

3.21 MiB

actual, against 1 GiB reserved — the reservation is dirty anonymous memory, allocated up front

STORAGE

3 KiB

actual, against 1 GiB reserved. Sparse files mean logical size is not physical cost

STATUS

What landed, and what did not

A roadmap that only lists wins is a brochure.

LANDED

- Contracts and the OpenAPI surface
- Sidecar: REST, MCP, exec, stats
- Control plane: databases, rotate, upgrade, costs
- Infrastructure: Container Apps, APIM, VNet
- Private endpoints for Blob, NFS and ACR pulls
- Entra ID management auth and the PKCE browser flow
- The browser terminal

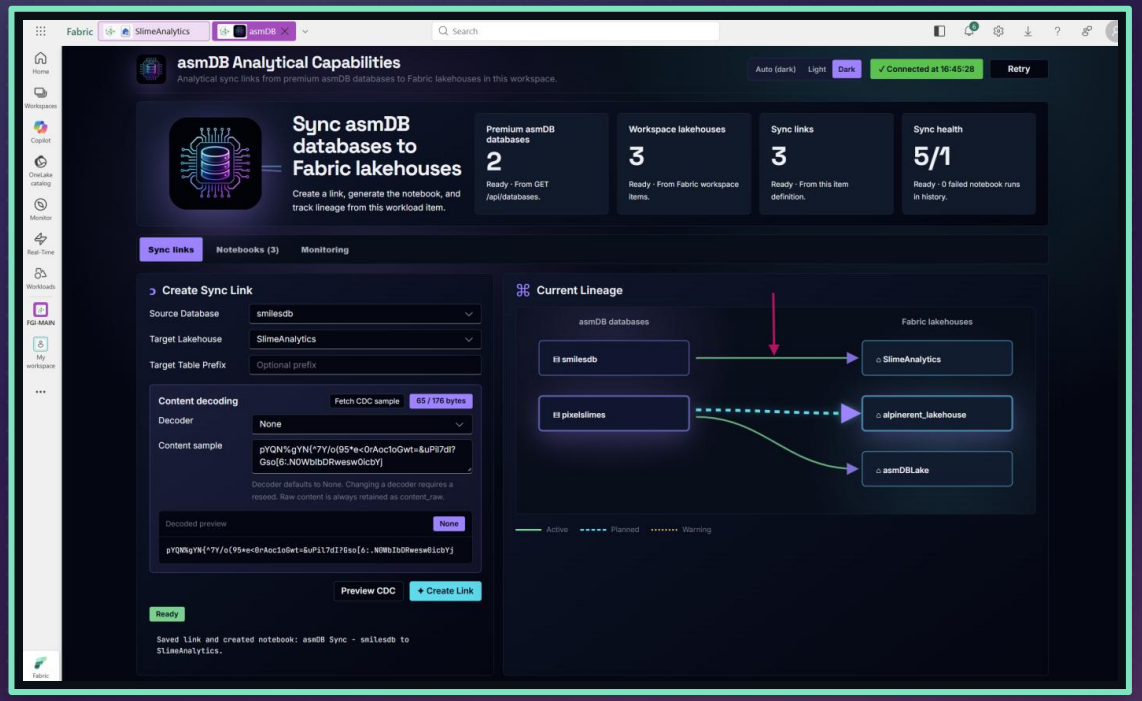
NOT LANDED

- Invoice-grade metering
- Backup and point-in-time restore beyond the NFS snapshot and the pre-upgrade BACKUP
- A build agent inside the VNet
- Read replicas and failover
- Automated share growth
- Preallocation economics

03

THE WORKLOAD

asmdb.analytics



A Microsoft Fabric workload that turns an assembly engine's change log into Delta tables in a customer's Lakehouse — without ever writing them itself.

THE PROBLEM

A bounded OLTP engine is the wrong place to ask questions

3,145,728 rows of 256 bytes is a fine operational store. It is a terrible analytical one — and making it a good one would ruin it.

WHY NOT IN THE ENGINE

No SQL
You cannot GROUP BY over a line protocol that only knows FIND and RANGE.

No columns
The payload is an opaque blob. There is nothing for a query engine to prune or push down.

One writer
An analytical scan competing with the write path is a scan that hurts production.

Fixed capacity
History has to leave the database eventually, or the database stops accepting writes.

WHY IN THE LAKEHOUSE

Delta on OneLake
Columnar, statistics, file skipping, time travel — all the things the engine refuses to implement.

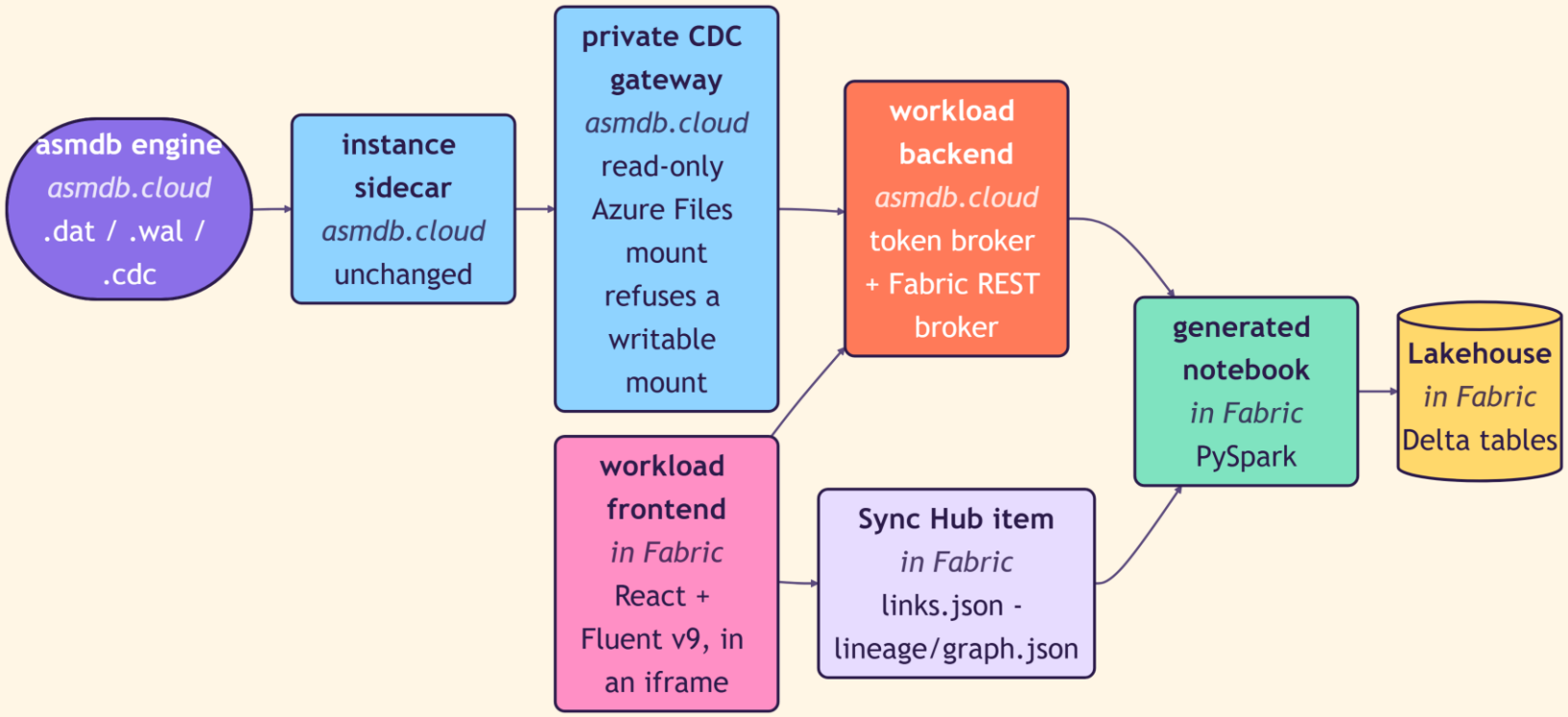
Fabric Spark
Compute that scales independently of the instance, and stops costing money when idle.

Direct Lake
A semantic model reading Parquet without a copy and without an import refresh.

Unlimited readers
Analysts, notebooks and reports never touch the single writer.

What a Fabric workload actually is

A frontend that runs in an iframe inside Fabric, a backend that brokers tokens, an item definition that stores your links, and a notebook that does the work.



Five moving parts

FRONTEND
React and Fluent UI v9, hosted by us, rendered inside Fabric.

BACKEND
Exchanges a Fabric JWT for scoped asmdb Cloud credentials. No service identity fallback.

SYNC HUB ITEM
A first-class Fabric item. links.json and lineage/graph.json are its definition.

CDC GATEWAY
A small Go service on a read-only mount, inside our VNet only.

NOTEBOOK
Generated PySpark, owned by the customer's workspace.

Read-only by construction, not by policy

The most dangerous component in this design is the one that reads another tenant's data files. So it was built so that it cannot do anything else.

It refuses a writable mount

On startup the gateway checks that its Azure Files mount is read-only. If it is writable, it does not start. A misconfiguration becomes a failed deployment, not a silent risk.

It stores nothing

No cache, no spool, no temp file. Frames are streamed from the change log to the caller. There is no copy of customer data to leak, and nothing to purge.

It is not on the internet

The gateway is reachable only from inside the service VNet. Fabric never talks to it directly — the notebook goes through the brokered backend.

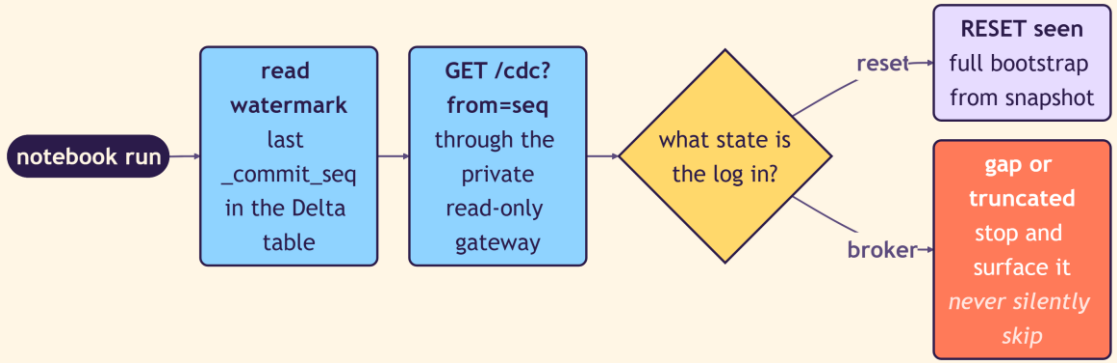
The one sentence that settles every design argument

Fabric Spark writes the Delta tables. asmdb.cloud does not. We publish a change log and generate a notebook; the customer's own capacity, in the customer's own workspace, does the writing. That keeps the data in their tenant, the cost on their capacity, and the blame in the right place.

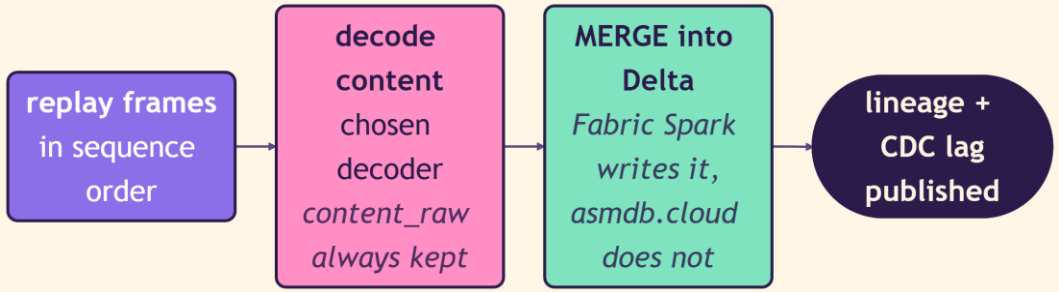
Read the watermark, replay the frames, merge

The interesting part is not the happy path. It is what happens when the log is not what the notebook expected.

READ — never trust, always verify



WRITE — Fabric Spark owns the Delta table

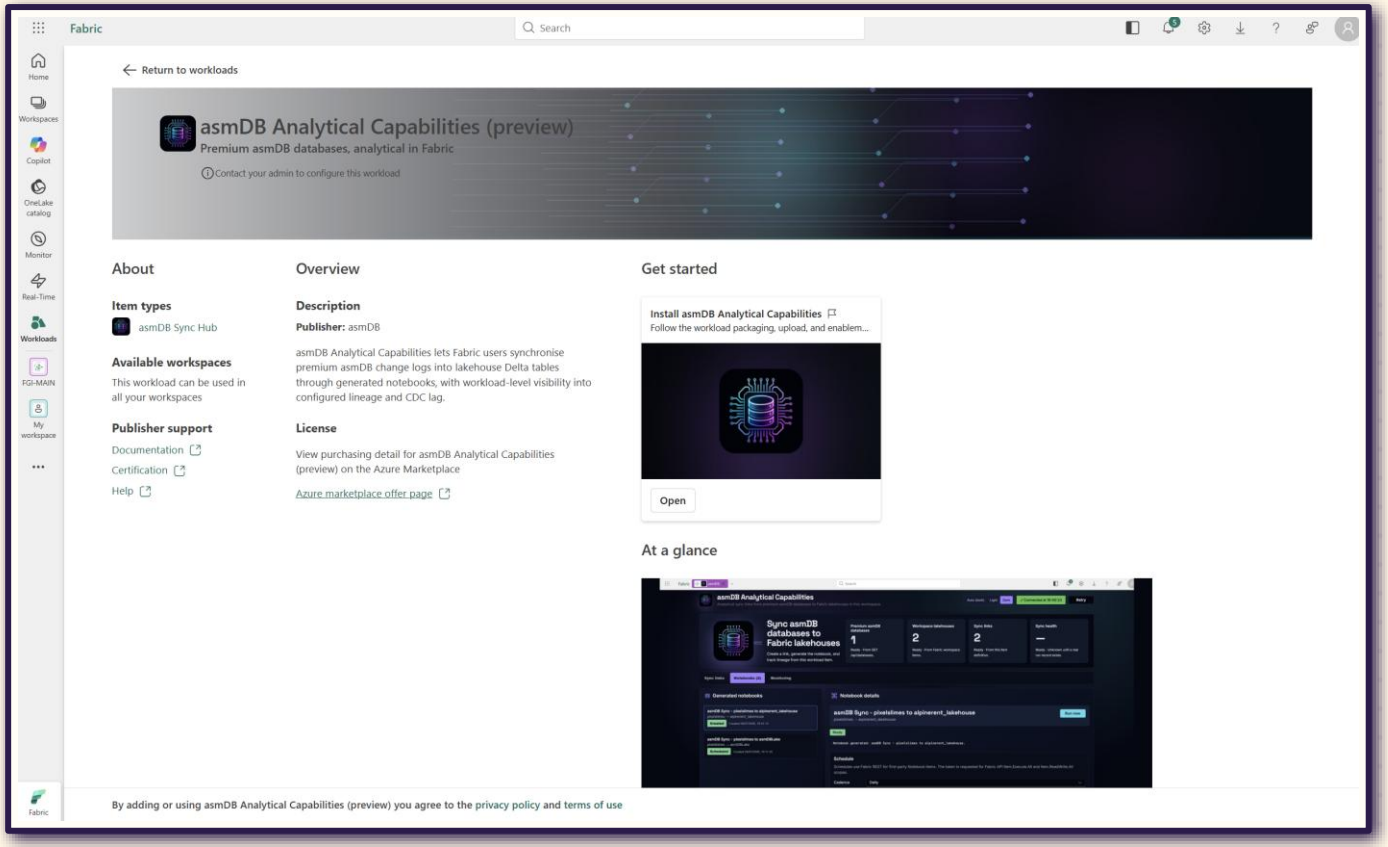


Never silently skip

A gap in the sequence stops the run and shows up as a failed sync, not as a table that is quietly missing yesterday.

It looks like any other Fabric workload, because it is one

Published through the Workload Development Kit: an item type, a publisher, a marketplace offer, and an admin who has to enable it.



ITEM TYPE
asmDB Sync Hub

PUBLISHER
asmDB

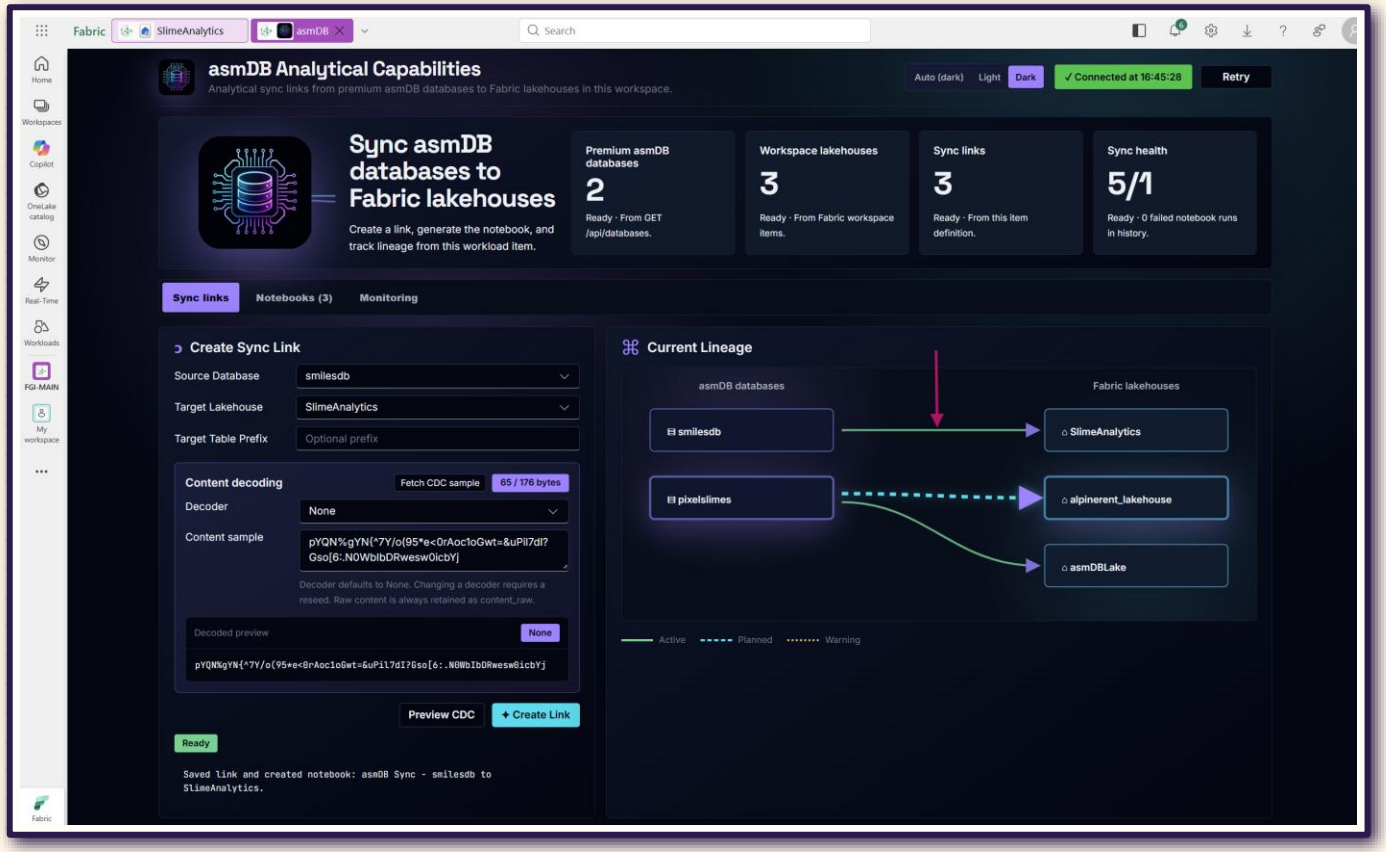
AVAILABILITY
All workspaces in the tenant, once an admin enables it

LICENSING
Through the Azure Marketplace offer page

DESCRIPTION
Synchronise premium asmDB change logs into Lakehouse Delta tables through generated notebooks, with workload-level visibility into lineage and CDC lag

Source database, target Lakehouse, and a decoder

The decoder is the interesting field. content_raw is always kept, so choosing a decoder is never destructive — but changing one forces a reseed.



PREMIUM DATABASES
2
from GET /api/databases

WORKSPACE LAKEHOUSES
3
from Fabric workspace items

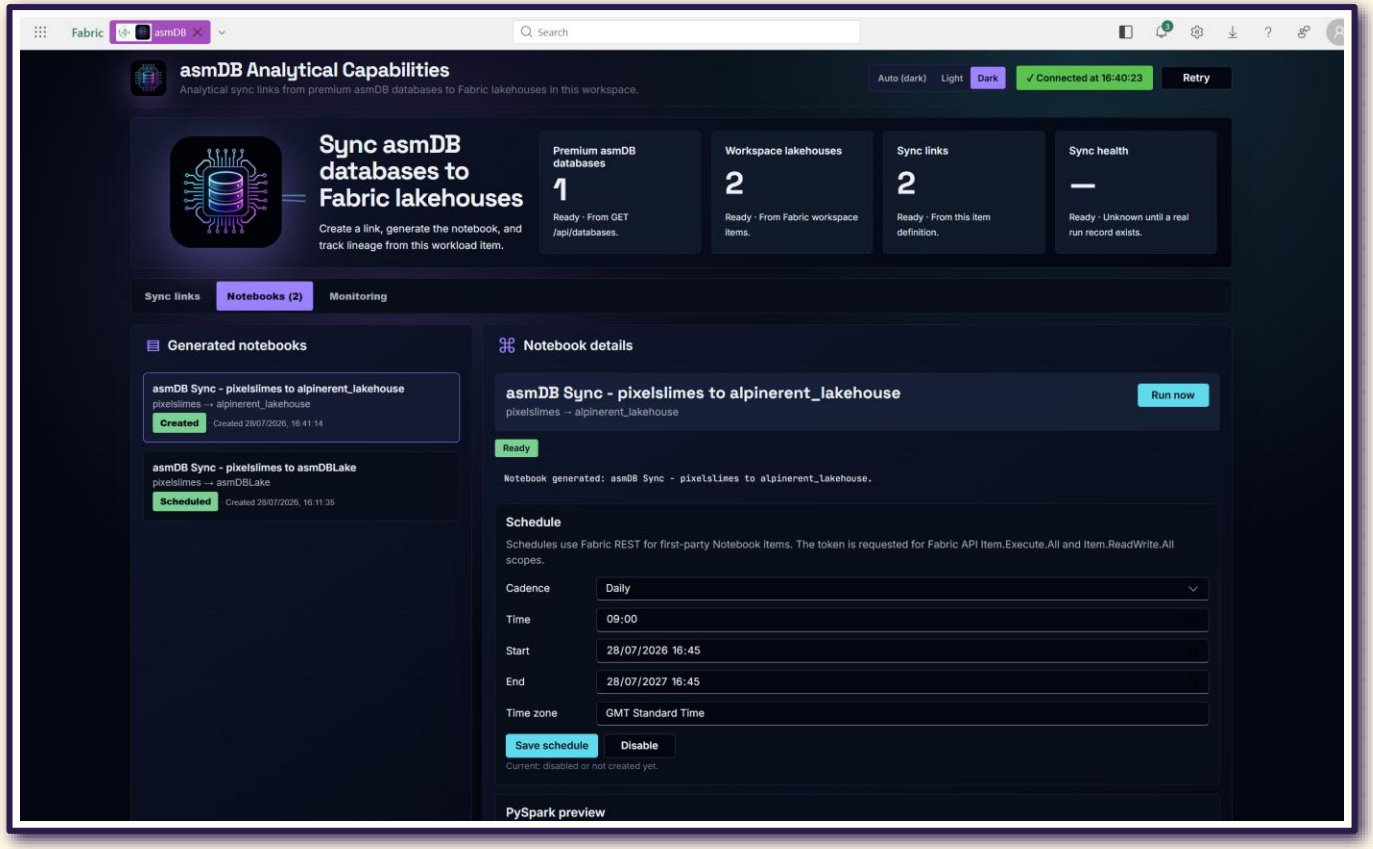
SYNC LINKS
3
from this item definition

SYNC HEALTH
5 / 1
0 failed notebook runs in history

GENERATED CODE

The workload writes the notebook, the workspace owns it

Schedules use the Fabric REST API for first-party Notebook items, with the token requested for Item.Execute.All and Item.ReadWrite.All.



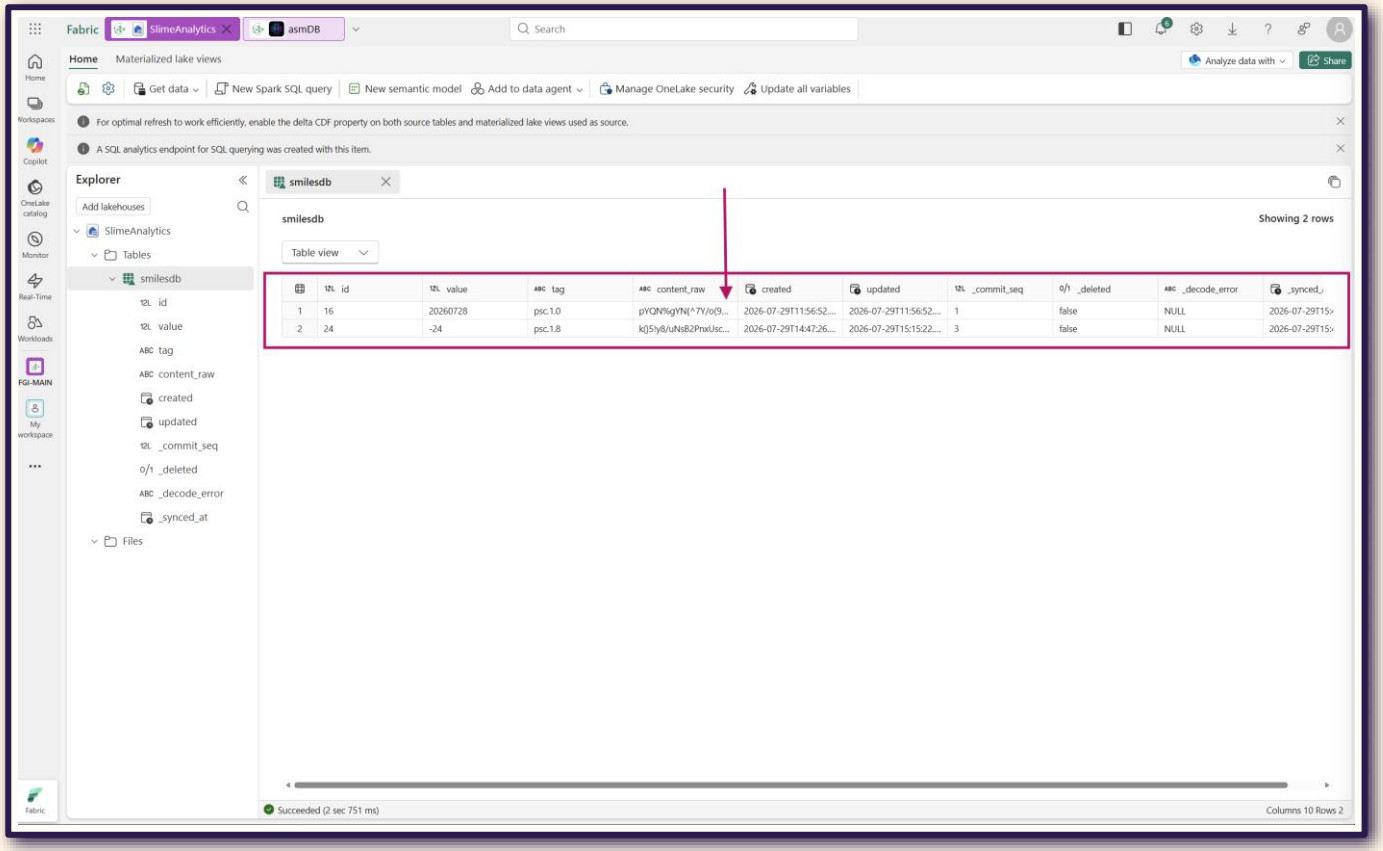
Why generate, not run

- The customer can read every line before it touches their data.
- It runs on their capacity, under their governance, with their identity.
- They can edit it, fork it, or schedule it from a Data Factory pipeline instead.
- If we disappear tomorrow, the notebook still works.

```
Convenience: native schedule, human identity.
Production: Data Factory pipeline, Workspace Identity.
```

What an assembly row looks like as Delta

Four columns from the engine, five added by the sync. The raw payload is never thrown away.



Columns

id	engine
value	engine
tag	engine
content_raw	engine
created	engine
updated	engine
_commit_seq	sync
_deleted	sync
_decode_error	sync
_synced_at	sync

Lineage and lag, derived from what actually ran

Link state is not a status field somebody sets. It is computed from Fabric job history.

ACTIVE

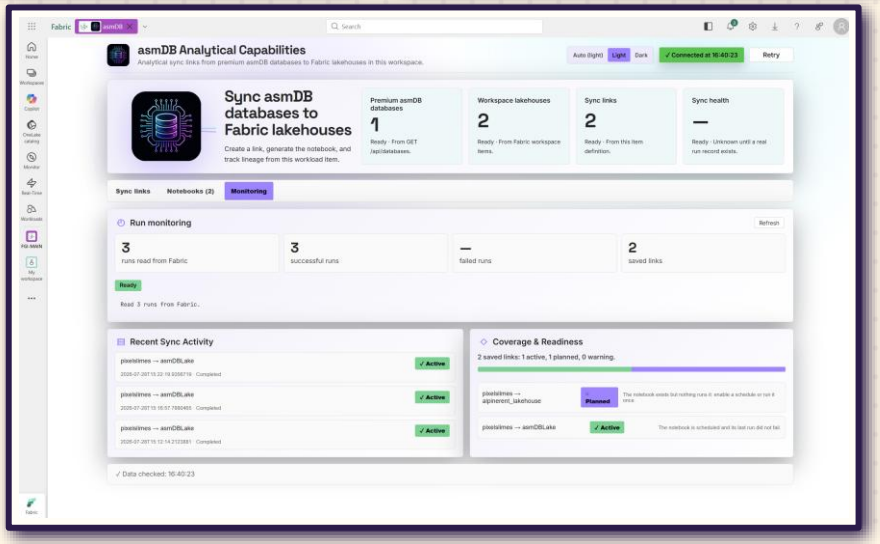
A notebook run has succeeded, and the watermark is moving.

PLANNED

The link and the notebook exist, but nothing has run yet. Shown as a dashed edge, never as a success.

WARNING

Runs are failing, or the CDC lag has grown past what the schedule should allow.



Sync health, honestly

When there is no run history at all, the tile says “Unknown until a real run record exists” rather than showing a green tick. A dashboard that reports success before anything has happened is worse than no dashboard.

DECODING

Where opaque bytes become meaning

The engine stores 175 bytes of text and has no opinion about them. The workload gives you a place to say what they are — without ever losing the original.

```
content decoding

content_raw   pYQN%gYN{^7Y/o(95*e<0rAoc1oGwt=&uPil7dI?Gso[6:.N0WbIbDRwesw0icbYj

decoder       None  →  Z85  →  PSC-1
sample size   65 / 176 bytes

; content_raw is ALWAYS retained.
; changing a decoder requires a reseed, because the
; decoded columns are derived, not authoritative.
```

The default is None

Doing nothing is the safe default. A sync that lands unreadable bytes is recoverable; a sync that lands wrongly-parsed bytes is not.

The decoder is the tenant's

asmdb has no idea what PSC-1 is. PixelSlime does. The workload just carries the bytes and lets the owner name them.

Failure is a column

_decode_error survives into the Lakehouse, so “how many rows did we fail to understand” is a query, not an incident.

04

THE PRODUCT

www.pixelslime.cloud

One AI-generated slime a day, forever. A whole trading card packed into 175 bytes of asmdb, and fingerprinted on Polygon so it cannot be quietly rewritten.



WHY

Proof by product, not proof by benchmark

A benchmark tells you an engine is fast. A product tells you it is correct, durable, operable, and worth paying for.

It writes every day

Not a load test that runs for an hour. A scheduled job that has to succeed at 10:00 Europe/Paris, unattended, or the site is visibly wrong.

It cannot lose data

Each card is unique and unrepeatable. There is no “regenerate it”. If asmdb loses a row, the loss is public and permanent.

It is externally verifiable

The bytes in the database are hashed on a public chain. If storage silently corrupted anything, the hash would stop matching and anyone could check.

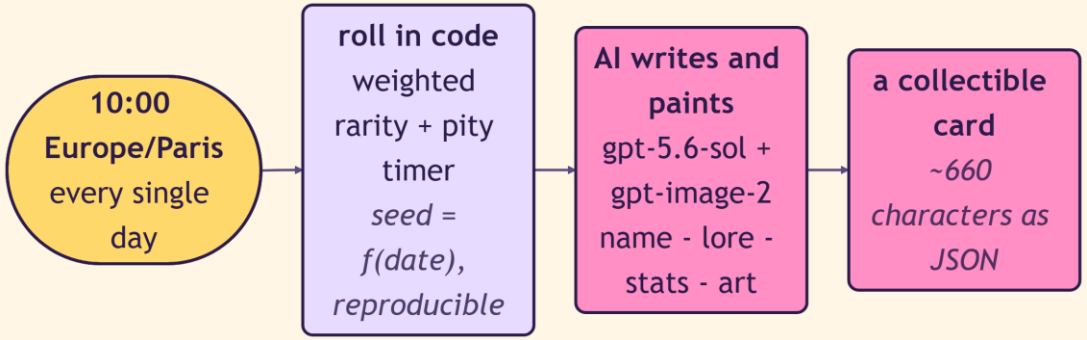
It produces analytics demand

A daily stream of encoded rows is exactly the workload the Fabric workload was built to move — so the last two layers get tested too.

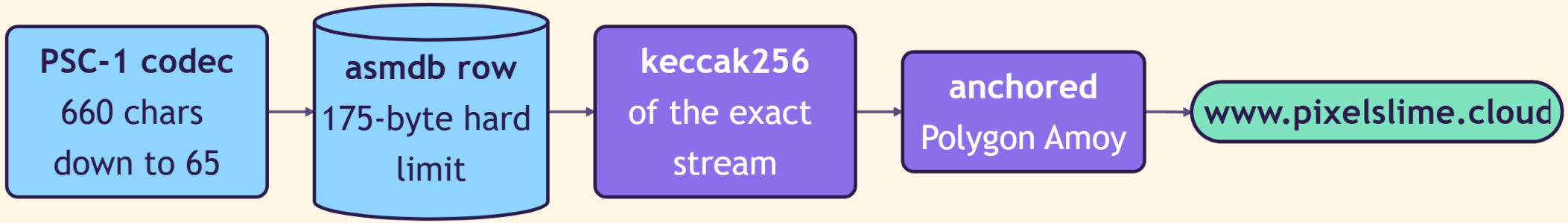
Make it, keep it, prove it

The rarity roll happens in code with a seed derived from the date, so the outcome is reproducible and the model cannot cheat.

MAKE — one card, invented from nothing



KEEP IT — and prove it



The rarity is rolled in code, never by the model

An LLM asked to "pick a rarity" produces a distribution nobody can audit and nobody can reproduce. So it never gets asked.

COMMON	*Puddle*	45 %	plain wooden frame, soft palette
UNCOMMON	*Dewdrop*	27 %	silver frame, a light halo
RARE	*Prism*	17 %	blue frame, prismatic highlights
EPIC	*Aurora*	8 %	gold and violet frame, a crown gem
LEGENDARY	*Starlight*	2.5 %	celestial frame, constellation backdrop
MYTHIC	*Dreamdrop*	0.5 %	rainbow foil, a frame that overflows itself

Reproducible

The seed is a function of the date, so the same day always rolls the same card. The whole collection can be replayed from scratch and audited against the published weights.

The pity timer

Fourteen days without a card of RARE or better, and the next roll is forced to be at least RARE. At one card a day, pure randomness would eventually produce a stretch nobody would stay for.

Data first, image second

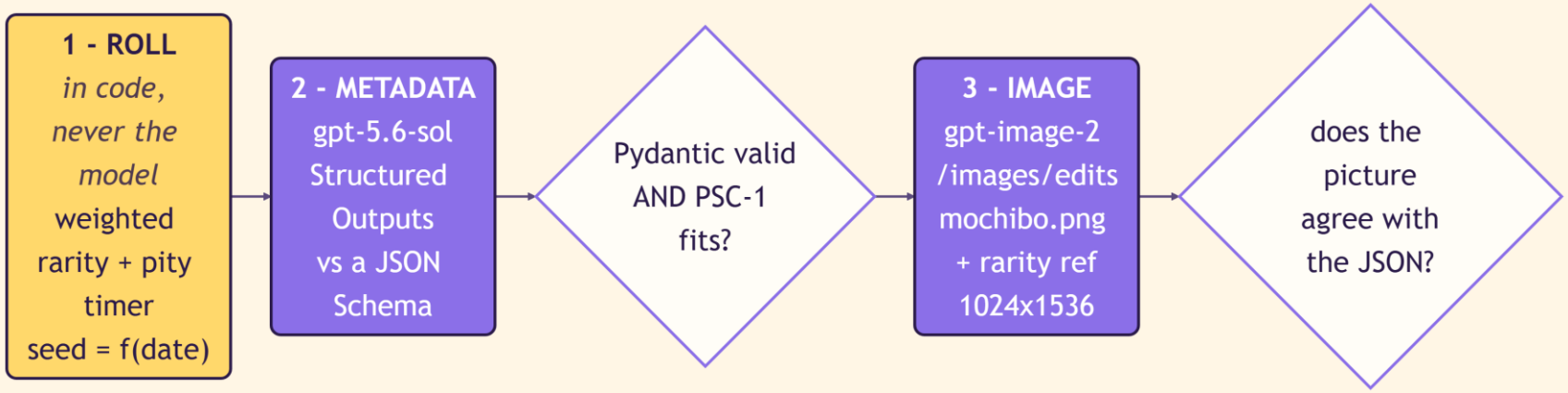
The obvious approach is to generate a beautiful card and then read the stats back off it. That approach is fragile, and it puts the model in charge of the database.

What we did not do

Paint a card, then OCR the name, level, rarity and four stat bars out of the pixels. Every card becomes a parsing problem, every font quirk becomes a data bug, and the database ends up believing whatever the renderer happened to draw.

What we did instead

Generate a structured JSON first, ask gpt-image-2 to paint exactly that JSON, then verify the image agrees with it. The data is authoritative and the artwork is a rendering of it — never the other way round.



Gate one retries the metadata up to twice · gate two repaints once · after that the day fails loudly rather than quietly

One model writes the facts, the other paints them

Neither is trusted with the other's job, and neither is trusted with a number the database will have to live with.

gpt-5.6-sol — the metadata

STRUCTURED OUTPUTS
Constrained to a JSON Schema, so the response cannot be prose, cannot miss a field and cannot invent one.

THE PROMPT
A versioned master prompt, plus the values already rolled in code, plus every name used so far.

LENGTH-CAPPED
Every field has a maximum, because the whole card has to fit in 175 bytes.

gpt-image-2 — the artwork

`/images/edits, NOT /generations`
Multipart, because only the edits endpoint accepts reference images at all.

`image[] = mochibo.png`
The anatomy canon, sent on every single generation, so 3,650 cards share one layout.

`image[] = ref-<rarity>.png`
An approved exemplar of the tier that was rolled, so rarity stays visible.

The wrinkle that made this interesting

Mochibo is an EPIC card — ornate gold and violet, gem at the crown. Use it as the only reference and a COMMON slime inherits an EPIC-looking frame, so rarity quietly stops being visible, which is the one thing a collectible cannot afford. The fix was not to stop sending Mochibo, but to split what the reference carries: the anatomy is pinned by the image, the finish is pinned by words and by a per-tier exemplar. Both are versioned.

The API said no, so Pillow said yes

A cute engineering detour that turned out to be the most reusable code in the pipeline.

the blocker

```
POST /openai/v1/images/edits  model=gpt-image-2  background=transparent

400 {"message":"Transparent background is not supported
                for this model.",
     "param":"background","code":"invalid_value"}

# accepted on /generations, rejected on /edits — and /edits is the
# only endpoint that takes the reference card at all.
```

The fix

Ask the model for a flat solid white exterior, then recover real alpha with Pillow by flood-filling inward from the four corners.

That is the exact technique already written to strip the checkerboard off the reference card — the tool built to clean the input turned out to be the tool that produces the output.

Seeded per corner

The fill starts from each corner's own colour and follows only connected pixels, so whites inside the card — clouds, text, highlights — are protected by the darker frame between them and the edge.

A leak guard

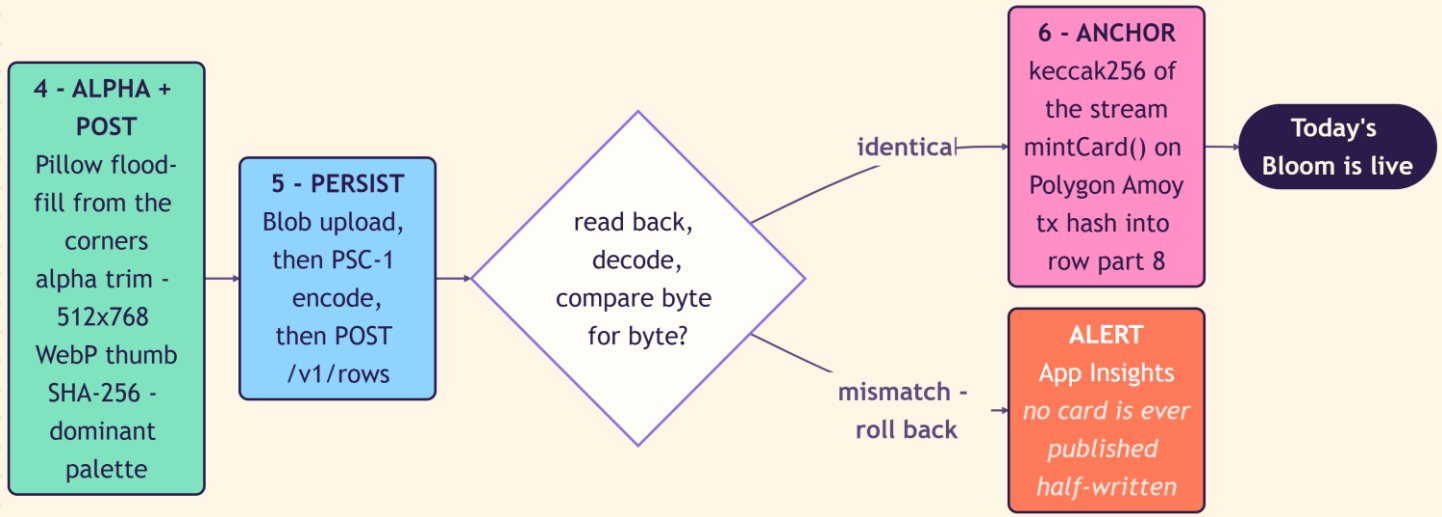
Any fill that consumes more than 75 % of the canvas is rejected outright. A flood fill on artwork is not risk-free, and a runaway fill would silently eat the slime.

Pillow decides, not the model

The authoritative transparency check is the corner test in code. The vision model disagreed with itself across two test cards, because its answer depends on how its viewer composites alpha.

No card is ever published half-written

Four checks, each of which can stop the day's card. A missing bloom is embarrassing; a wrong one is permanent.



Why the vision gate is not the last word

A model checking another model's work is useful, but it is not evidence. So the only gate that can actually publish a card is the deterministic one: decode the bytes that landed in the database and compare them with the bytes that were sent.

GATE 1 · SCHEMA
Pydantic validates the JSON and a trial PSC-1 encode has to fit. Up to two retries, then the day fails loudly.

GATE 2 · VISION
A model looks at the painted card and answers: do the name, level, rarity and stats match the JSON? Is there alpha? Is this a Mochibo clone?

GATE 3 · READ BACK
The rows are read out of asmdb, decoded, and compared byte for byte with what was sent. A mismatch rolls the whole thing back.

IDEMPOTENCY
The business key is the date. The job opens with GET /v1/range?lo=today&hi=today and stops if a card exists, so a retry or a redeploy cannot mint twice.

Meet Mochibo — PS-0001

Everything on this slide lives inside 65 characters of a 175-character column.



EPIC · AURORA | FAIRY | LV 12 | +760 $SMILE | ON-CHAIN #1

STRENGTH	28	ENDURANCE	55
AGILITY	65	HAPPINESS	95

THIS SLIME GENERATES **+760 $SMILE** happiness 95 × epic ×8 = 760

Minted fresh into the Claim Pool when this slime bloomed — while 100 $SMILE were burned from the Genesis Rain. Two different purses, on purpose.

```
id      16          tag  psc.1.0
value   20260728    (positive = a header row)
content pYQN%gYN{^7Y/o(95*e<0rAoc1oGwt=&uPil7dI?Gso[6:.N0WbIbDRwesw0icbYj
```

THE CONSTRAINT

The 175-character wall

asmdb's content column is UTF-8 text, at most 175 bytes, with no NUL, CR or LF. That is not a suggestion. Everything else in PixelSlime is a consequence.

A CARD AS JSON

~660

characters, formatted the obvious way

AFTER PSC-1

65

characters for Mochibo — a 10× reduction

THE COLUMN

175

characters, hard limit, enforced by the engine

PER ROW, BINARY

140

bytes, because Z85 turns 4 bytes into 5 characters

Why not just widen the column?

Because the column cannot be widened. The record is 256 bytes, and after the id, the status byte, the flags, two microsecond timestamps and two length fields, 175 bytes are what remain for content. Widening it would mean changing the record size, which would change the address arithmetic, which would change the on-disk format. The constraint is not a product decision that can be revisited — it is the engine's geometry. So PixelSlime was designed to fit it, and the codec is the interesting part of the product precisely because of that.

A 32-byte header, little-endian, fixed layout

Nothing here is self-describing. Every field is at a known offset, because a length prefix would cost bytes that do not exist.



One byte holds three things

Rarity, elemental type and the shiny flag are packed into byte 7 as a bitfield. Six rarities and sixteen types fit comfortably; spending three bytes on them would not.

Four bytes of the art hash

art_sha stores only the first four bytes of the artwork's SHA-256. Enough to detect that the image and the row have drifted apart; not enough to waste 28 bytes proving it.

The CRC is inside the budget

A CRC-16/CCITT-FALSE over the header catches corruption before decoding starts. Two bytes to avoid ever rendering a card built from garbage.

Compress against a dictionary you already agreed on

General-purpose compression is bad at short strings, because there is no room to build a table. So the table is shipped in advance.

encode

```
# five fields, joined by a unit separator
payload = "Mochibo\x1FSweet and cuddly\x1FSugar Squish\x1F..."

# raw DEFLATE, no zlib wrapper, preset dictionary
body = zlib.compress(payload, wbits=-15, zdict=DICT)

# 4 binary bytes become 5 printable characters
content = z85_encode(header + body)
```

THE PINNED DICTIONARY

3,215 B

shipped with the decoder, SHA-256 checked on every decode

PRIMING SAVES

35-50 %

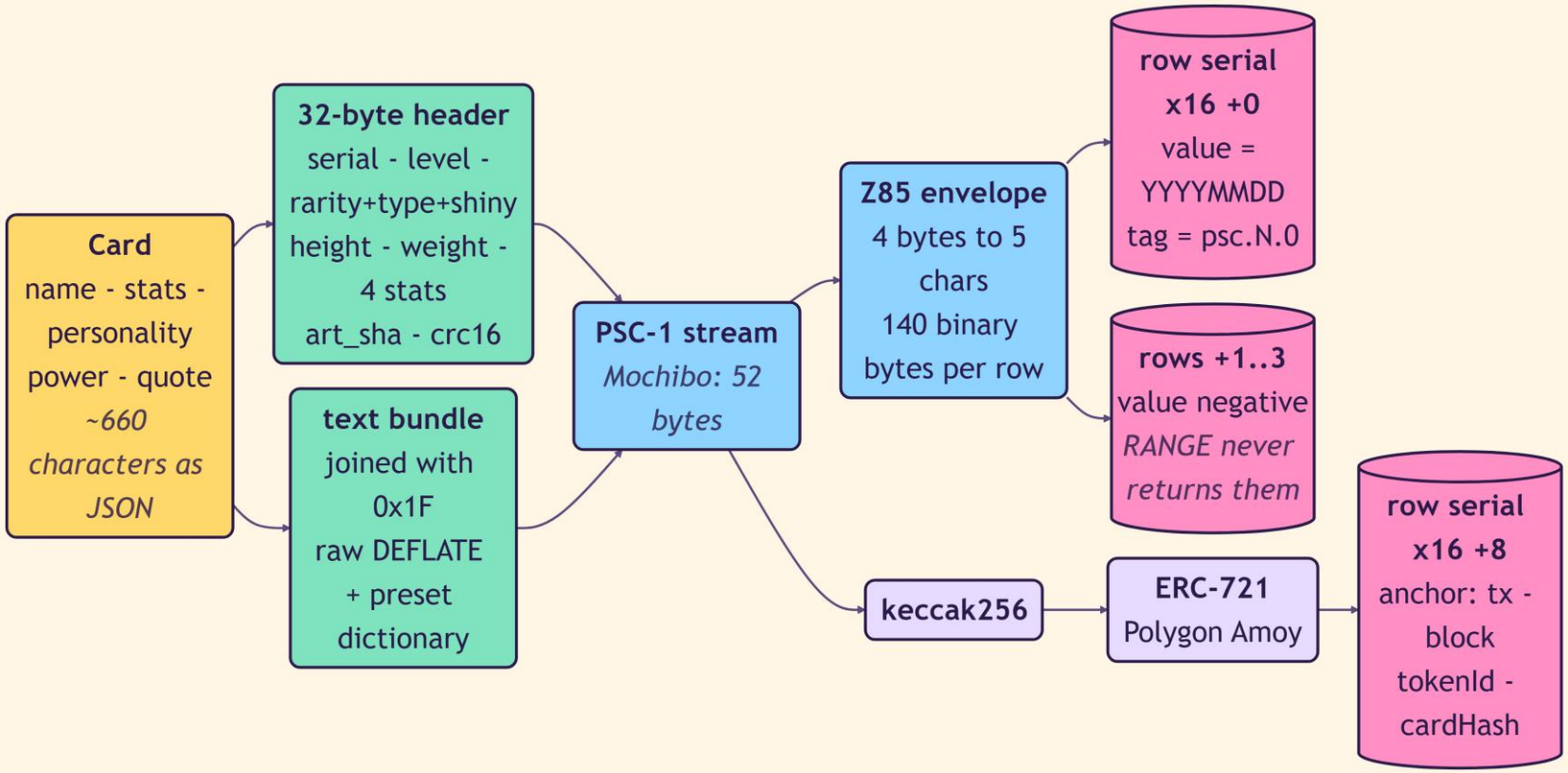
on short strings, versus DEFLATE with a cold table

The catch, and why it is worth it

A preset dictionary is a shared secret between the encoder and every decoder that will ever exist. Change one byte of it and every card written before the change becomes unreadable. That is why the dictionary is version-pinned, distributed with the decoder, and verified by hash on every single decode — a silent dictionary mismatch would corrupt the entire collection at once, and it would look like a rendering bug.

From a card to rows, and to a hash

The same stream is written to the database and hashed for the chain. There is exactly one source of truth for both.



Row addressing

row_id
serial × 16 + part

PART 0
the header row. value is the positive date, YYYYMMDD

PARTS 1–3
continuation. value is negative, so a RANGE scan over dates never returns them

PART 8
the anchor row: tx hash, block, tokenId and the full cardHash

MAX STREAM
560 bytes, four rows

Four contracts on Polygon Amoy

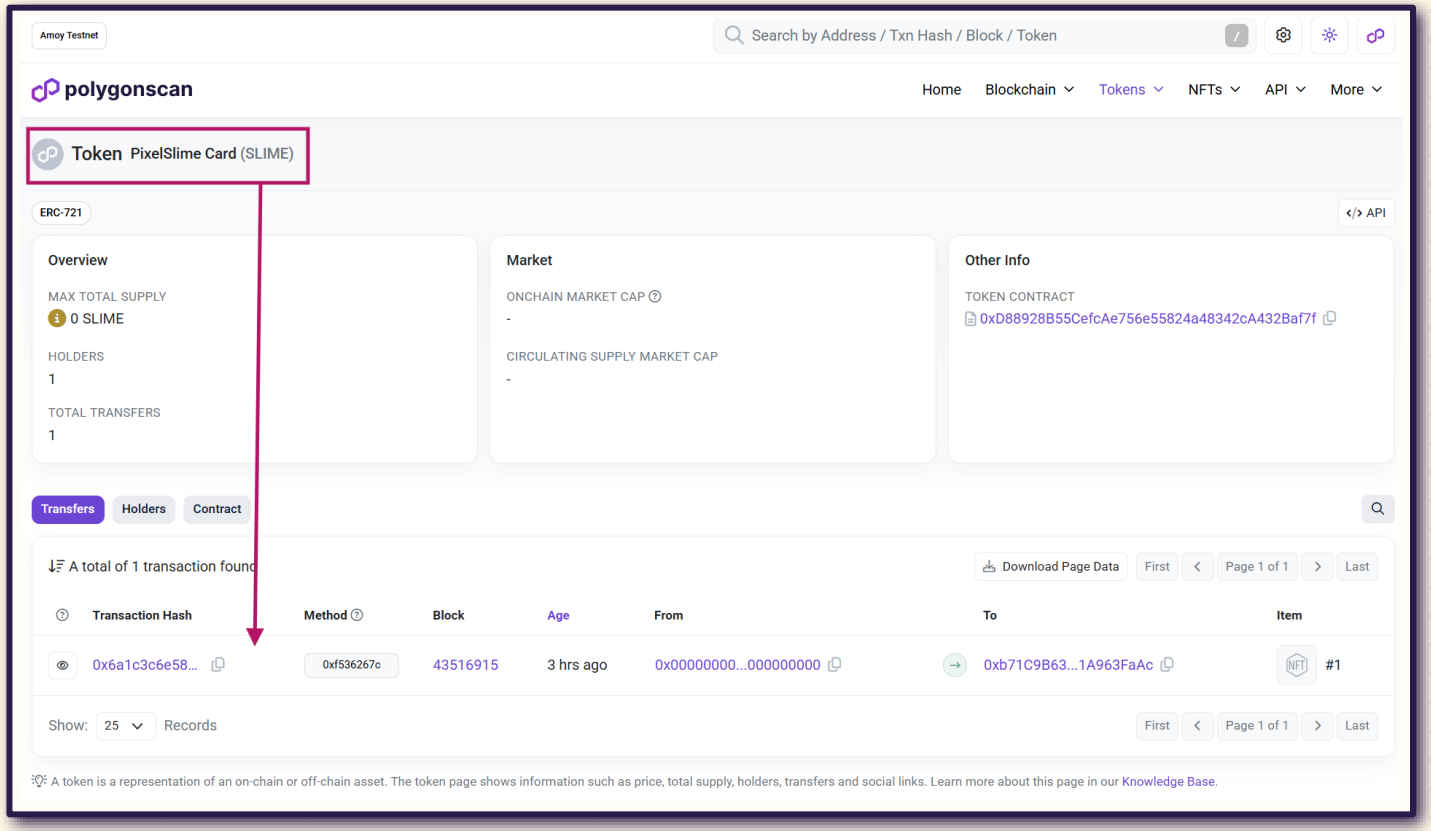
Chain id 80002. A testnet, deliberately — the point is verifiability, not speculation.

SmileToken	*ERC-20 · $SMILE*	`0x0BBaC39Bf418ab63BF71802808A4C63D4B39b798`
PixelSlimeCard	*ERC-721 · SLIME*	`0xD88928B55CefcAe756e55824a48342cA432Baf7f`
ClaimPool	*the only minter*	`0xbce1362c1155777df19F9cea6c8ECa68B155160d`
SlimeAdoption	*ownership transfer*	`0x20d6A4F365d00b4d27726f13093Dc2C497473CcA`

Guard rails that are in the contracts, not in a runbook: the treasury address and the admin address must differ; ClaimPool alone holds the minter role; and the signer is guarded so it can only ever act on a testnet.

Same bytes, same hash — or someone is lying

The keccak256 is taken over the PSC-1 stream itself, not over a JSON rendering of it. That is the whole trick.



1 · WRITE
The daily job writes the PSC-1 rows to asmdb.cloud and reads them straight back.

2 · COMPARE
It decodes what it read and compares byte for byte. A mismatch aborts before anything is published.

3 · HASH
keccak256 over the exact stream that is now in the database.

4 · MINT
mintCard(serial, cardHash, uri) on Polygon Amoy.

5 · ANCHOR
The transaction hash, block, tokenId and full cardHash are written back as row part 8.

Two tokens, one letter apart

SLIME is the card. SMILE is the currency. Conflating them is the single easiest way to misread everything that follows.

SLIME

ERC-721 · the card itself

WHAT IT IS
One non-fungible token per slime, holding the card's cardHash

HOW MANY
One per bloom. 3,650 will ever exist

WHAT IT PROVES
That this exact card existed on this exact day

SMILE

ERC-20 · the currency

WHAT IT IS
A fungible token minted into the Claim Pool by each bloom

HOW MANY
365,000 minted once as the Genesis Rain, never refilled

WHAT IT DOES
It is the thing that decrements — 100 burned per bloom



A ten-year clock made of arithmetic

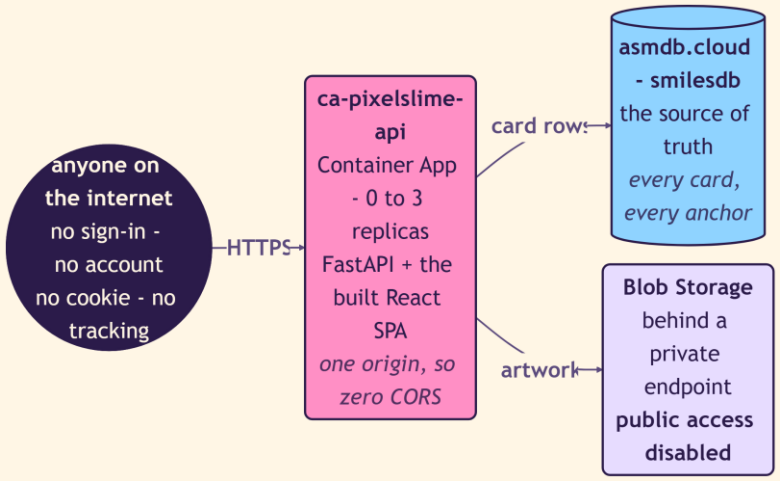
365,000 ÷ 100 = 3,650 blooms. One a day. The end date is not a policy anyone can change — it is a division that was fixed the moment the Genesis Rain was minted.

HOSTING

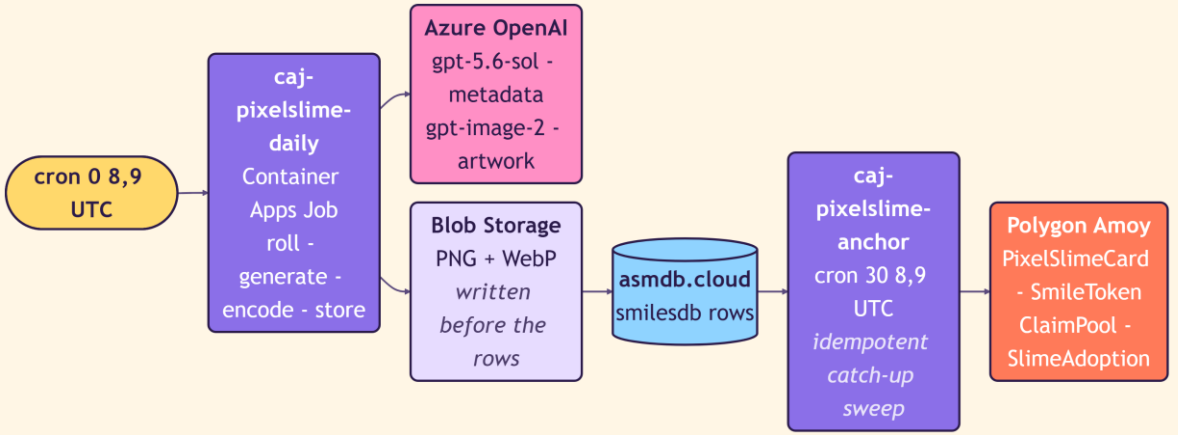
One Container App, two jobs, no account to create

The public site is anonymous by design: no sign-in, no cookie, no tracking. There is nothing to log into, so there is nothing to breach.

SERVING PATH



JOB PATH



One origin, zero CORS

FastAPI serves the built React bundle from the same Container App, so there is no cross-origin configuration to get wrong.

Idempotent by default

The anchor job is a catch-up sweep, not a one-shot. Running it twice is safe; missing a run is survivable.

A VNet forced by policy

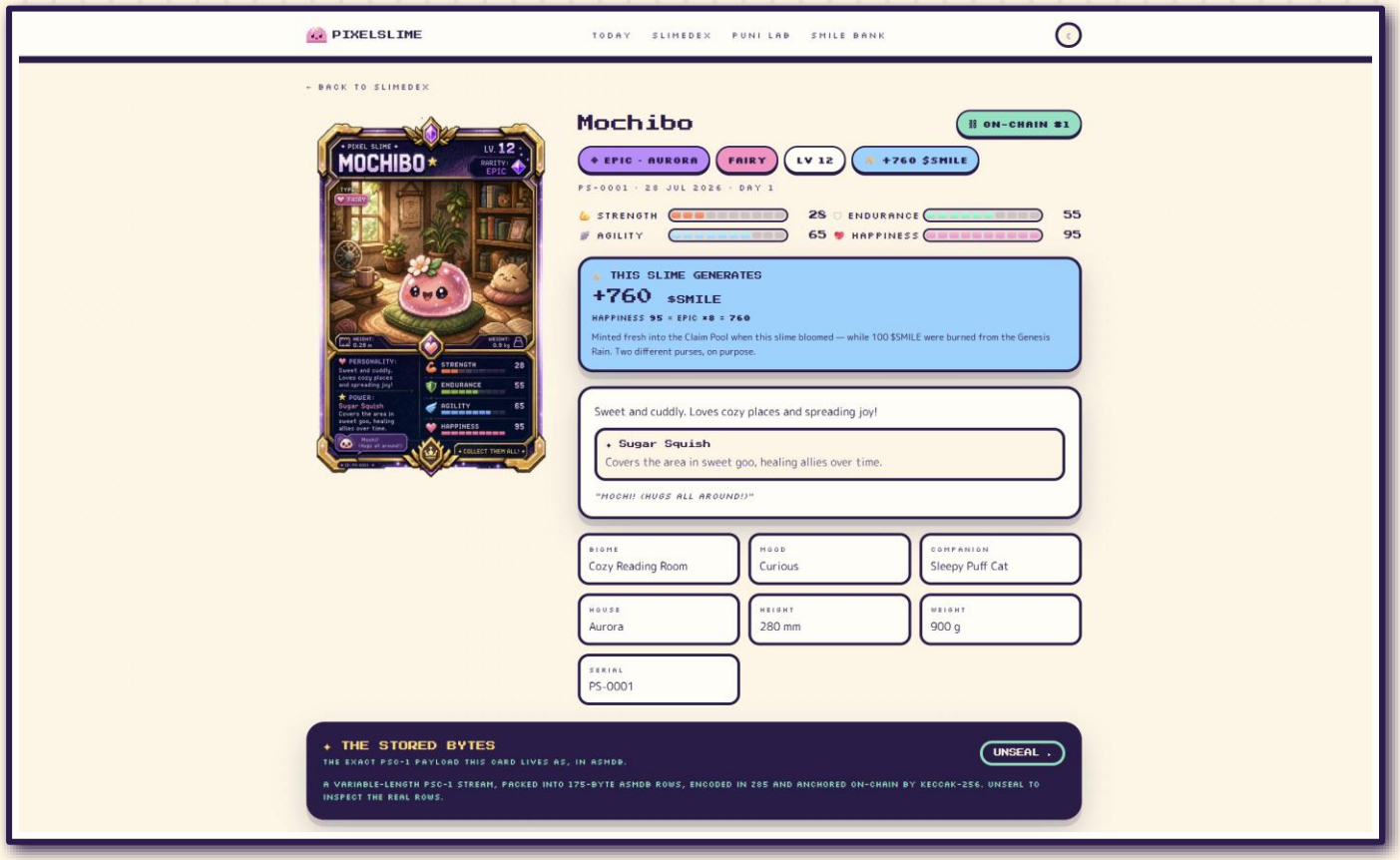
Storage and Key Vault had public access disabled by tenant policy, so private endpoints were not an aesthetic choice.

Nothing published half-written

Blob artwork is uploaded before the rows exist, and the rows are read back and compared before the card goes live.

www.pixelslime.cloud

Every number on this page was decoded, in the browser, from the row you saw two slides ago.



The Stored Bytes

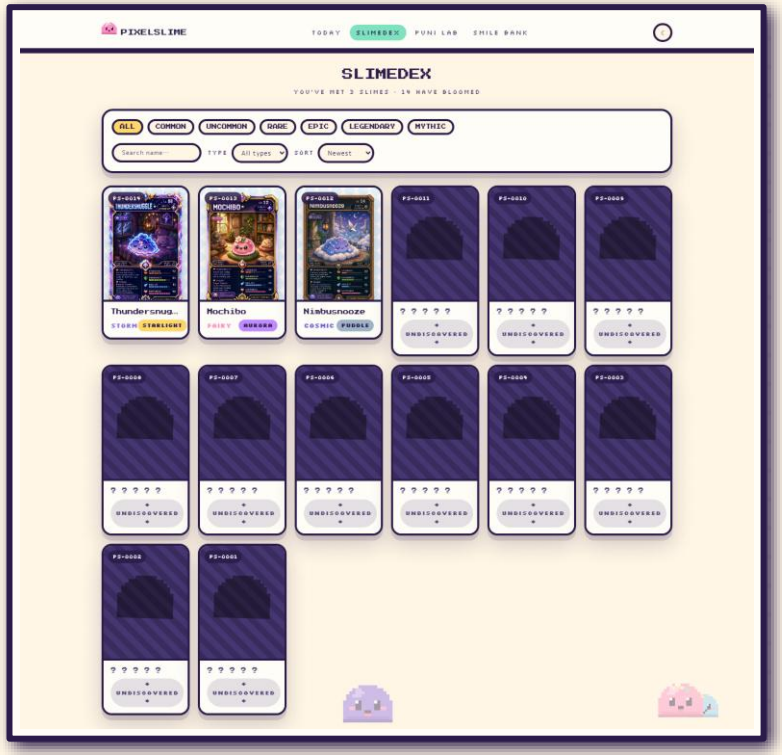
Every card page has an “UNSEAL” panel that shows the exact PSC-1 payload the card lives as, in asmdb. Not a pretty summary — the real rows.

It is the most important feature on the site and the least commercial. If a product claims its data is verifiable, it has to let you look.

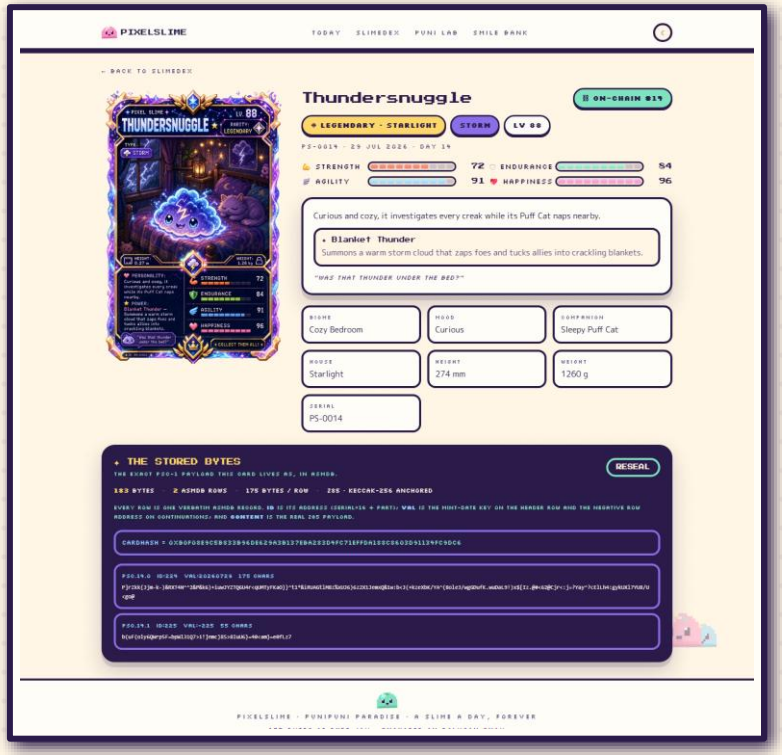
```
SLIMEDEX    the gallery
PUNI LAB    how it works
SMILE BANK  the economy
```

One a day, forever — and a page for every one of them

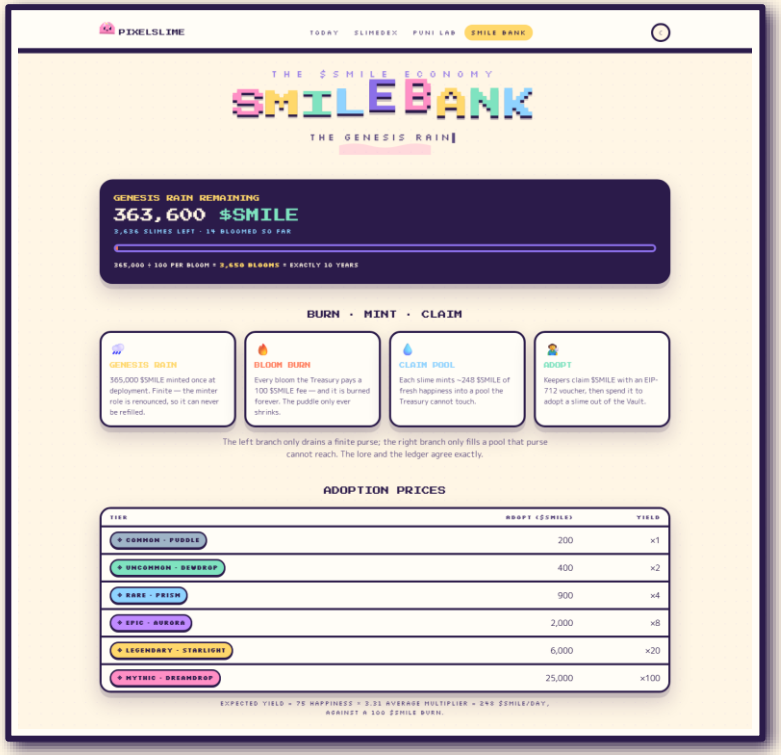
The SLIMEDEX, a card profile with its provenance, and the SMILE BANK. All three read from the same 175-byte rows.



SLIMEDEX · the gallery



PROFILE · provenance and raw bytes

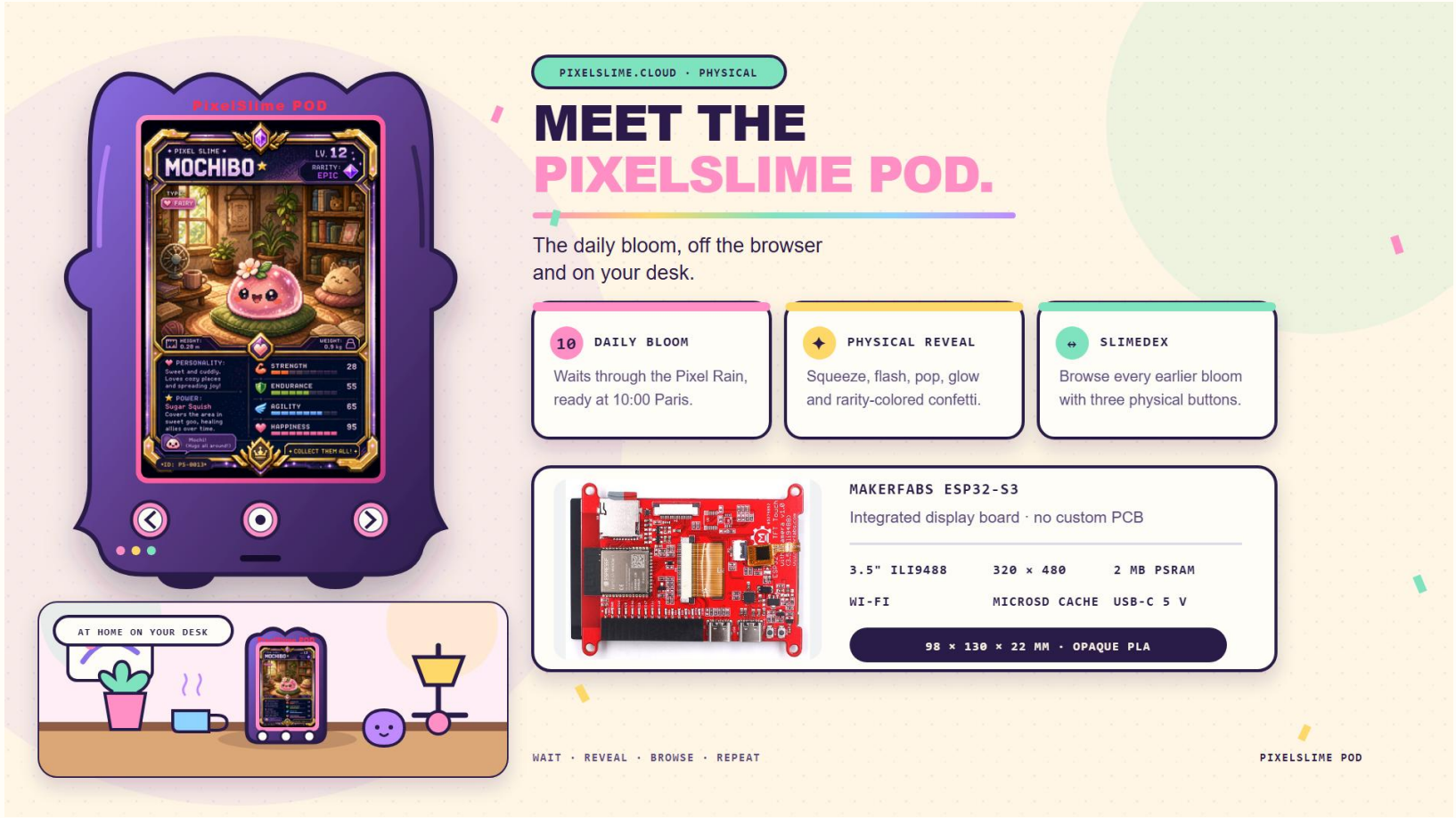


SMILE BANK · the ten-year clock

Anonymous, accessible and reduced-motion aware. There is a full Playwright screenshot suite for the reduced-motion variants, because "it works with animations off" is a claim that has to be tested, not asserted.

The daily bloom leaves the browser

A small ESP32-S3 display turns one API response into a desk-sized ritual.



A card that waits for you

The website publishes the bloom. The POD turns it into a physical moment.

PixelSlime POD: daily flow

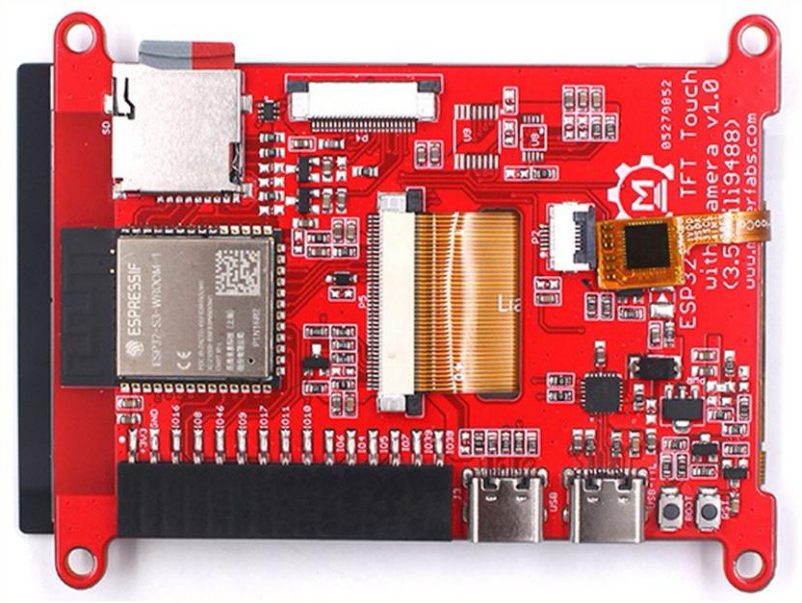
Four interface states designed for the 320 × 480 display



- Slow pixel rain, breathing silhouette, countdown
- Sealed card: no artwork before the center button is pressed
- Reveal: squeeze, flash, pop, 32 confetti pieces, rarity glow
- Left/right: history · center: details or return to Today

One board, three buttons, no custom PCB

The Makerfabs module already carries the display, touch, USB and microSD.



J3 / GPIO4 · GPIO5 · GPIO6

Normally-open buttons to common ground.
INPUT_PULLUP means no external resistors.

16 MB
FLASH

2 MB
PSRAM

320 × 480
PORTRAIT

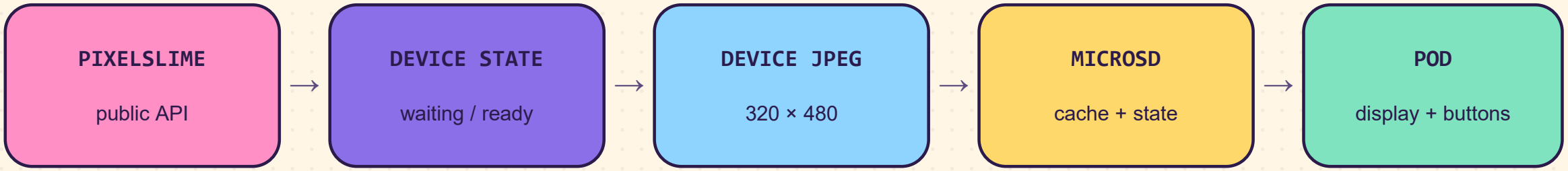
98 × 130 × 22 mm
ENCLOSURE

USB-C only. No battery, no charger, no custom power board.

The enclosure is opaque PLA: purple shell, red trim, removable rear cover.

The site already knows everything the POD needs

The hardware adds a new ritual, not a second source of truth.



Canonical art stays canonical

The 1024 × 1536 PNG and its PSC-1 art hash never change.

Reveal state is local

lastRevealedSerial prevents yesterday's ceremony replaying after reboot.

Offline means useful

Cached cards and the last synchronized countdown survive Wi-Fi loss.

Every card links home

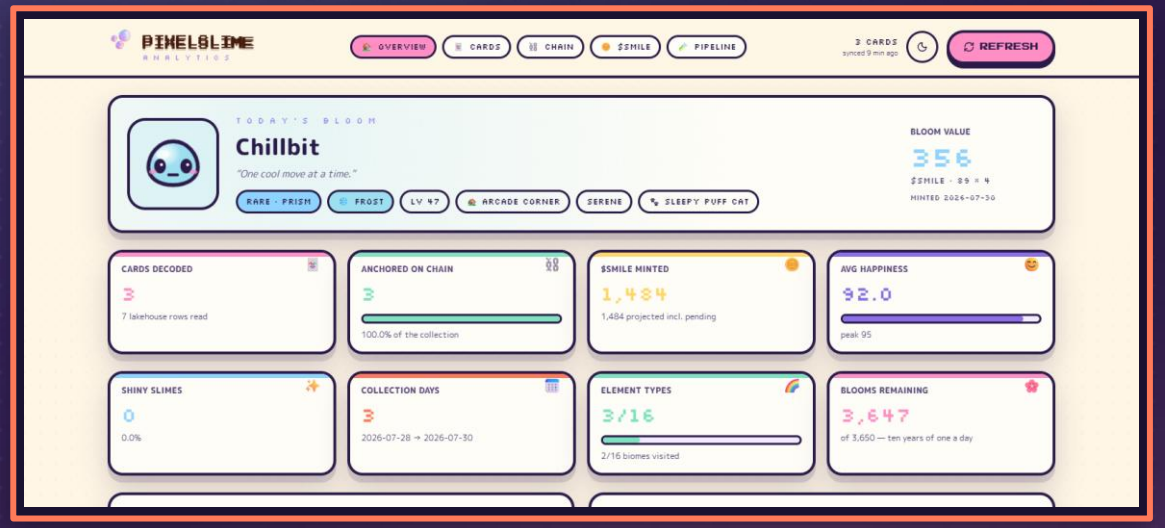
A generated QR opens the same public profile on pixelslime.cloud.

ONE COLLECTION · TWO SURFACES · ZERO DUPLICATED CARD LOGIC

05

THE PAYOFF

PixelSlime Analytics



A Fabric app that reads the Lakehouse, decodes PSC-1 back into cards, and answers the questions the engine was never designed to answer.

RAYFIN

A Fabric app in three commands

Rayfin generates the app backend and its SQL database from entity classes, deploys the static bundle, and wires up Fabric authentication.

rayfin

```
npx rayfin login
npx rayfin up           # deploy + apply schema changes
npx rayfin up status

npx fabric-app-data add semanticModel slimeRaw ...
npx fabric-app-data generate -o src/fabric.generated.ts

npm run dev             # Vite on :5173
npm run build:fabric    # the bundle Rayfin hosts
npm run test:fabric     # validated inside the portal
```

What it hands you

AUTH
Fabric authentication and embed auth, so the app single-signs-on inside the portal

DATA
A managed SQL database generated from the TypeScript entity classes in rayfin/data/

HOSTING
Static hosting of the built bundle, declared in rayfin.yml

BINDING
fabric.yaml registers the semantic model; fabric.generated.ts is derived from it

The app can only be validated inside the Fabric portal embed flow — so every UI change is checked with a real browser session against the real tenant, never against a local mock.

Five pages, one question each

Every page reads precomputed rows from the Rayfin database. None of them queries the Lakehouse directly.

OVERVIEW

What is in the collection right now?

Fourteen KPIs, rarity mix against the seeded roll table, element spread, blooms per day, biomes and moods.

CARDS

What does each individual card contain?

Search, filter, sort and a detail modal with stats, stream size, capacity used, art fingerprint and sync state.

CHAIN

Is what we claim actually on chain?

Anchored cards, awaiting anchor, average anchor lag, chain drift, and a ledger that links out to Polygonscan.

$SMILE

Where is the money going?

Minted, pending credit, burned, net supply change, Claim Pool, average yield per card and the burn runway.

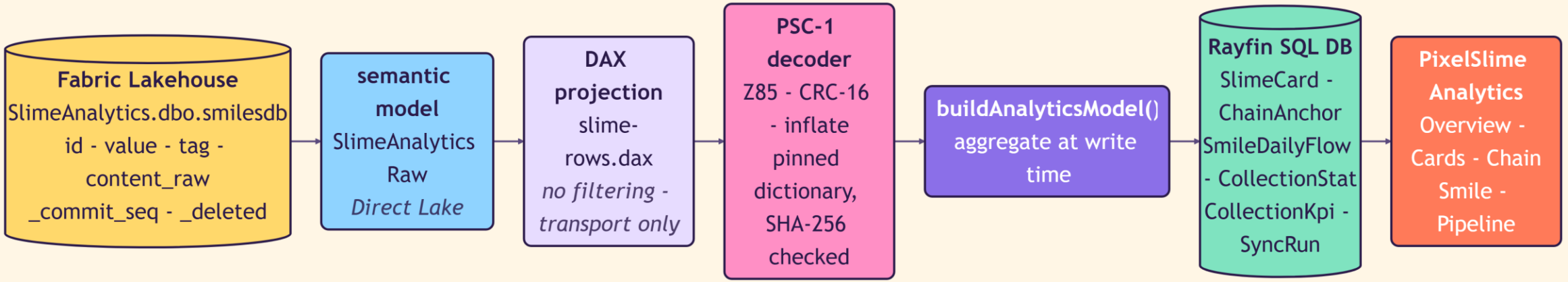
PIPELINE

How well is the encoding holding up?

Lakehouse rows read, total stream bytes, average capacity used, multi-row cards and storage pressure.

Direct Lake in, aggregated rows out

The semantic model is a transport, not the analytics layer. The DAX query does no filtering at all — the decoder needs every version of every row.



The lakehouse is the only source

There is no second write path, no admin override and no seeded fixture. If it is not in the Lakehouse, the app does not show it.

Aggregate at write time

The sync decodes, flattens and rolls up once, then persists. Pages read finished rows, so a page load never costs a decode.

Never trust flagOnChain

A card's own bytes can claim it is anchored. Only an anchor row proves it. Every chain number is derived from on-chain evidence.

Decode errors stay visible

A card that fails to decode is displayed as a card that failed to decode. It is never dropped from a count to make a chart look tidy.

Six entities, declared in TypeScript

rayfin up reads these classes and produces the SQL schema. The star schema is a side effect of the domain, not a modelling exercise.

SlimeCard

FACT

One row per decoded card: stats, rarity, type, stream size, capacity used, art fingerprint.

ChainAnchor

FACT

One row per anchor found on chain. Belongs to at most one SlimeCard.

SmileDailyFlow

FACT

Minted, burned and net change per day — the supply curve.

CollectionStat

AGGREGATE

Distribution rows: rarity mix, element spread, biomes, moods.

CollectionKpi

AGGREGATE

The scalar tiles on the Overview page, computed once at sync time.

SyncRun

AUDIT

One row per pipeline run, so “when was this last true” is always answerable.

SlimeCard has many ChainAnchor. ChainAnchor has an optional SlimeCard — because an anchor can exist for a card the decoder has not yet understood, and that asymmetry is the bug the Chain page exists to find.

THE NUMBERS

Fourteen KPIs, and none of them are decorative

Every tile is derived from decoded bytes or on-chain evidence. Nothing is a placeholder and nothing is rounded up.

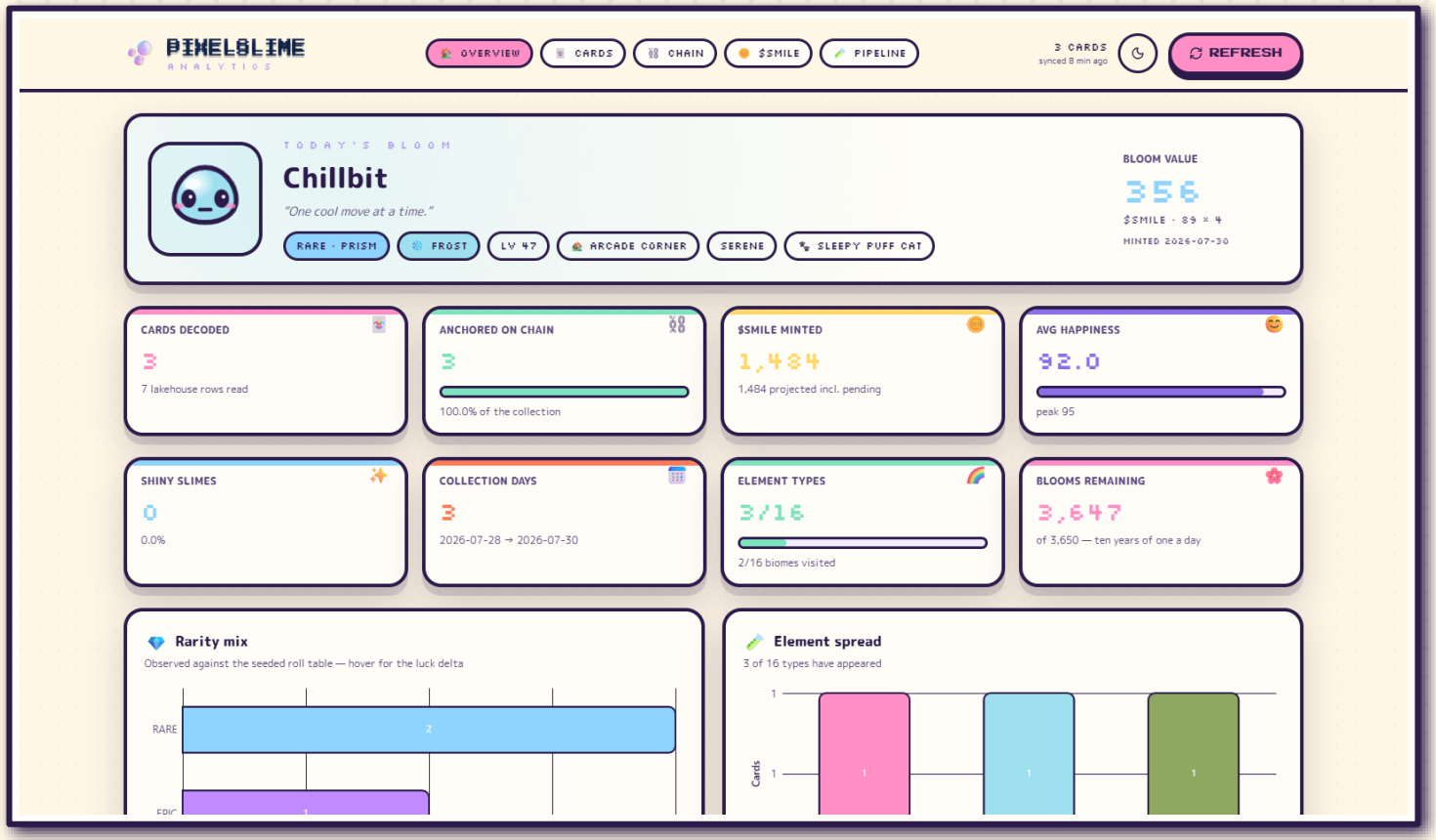
CARDS DECODED	ANCHORED ON CHAIN	$SMILE MINTED	AVG HAPPINESS
SHINY SLIMES	COLLECTION DAYS	ELEMENT TYPES	BLOOMS REMAINING
VERIFIED	SEED CARDS	WITH COMPANION	WITH ACCESSORY
DECODE ERRORS	CHAIN DRIFT		

Chain drift — the metric that admits failure

Chain drift counts cards whose own bytes claim they are anchored while no anchor row exists. It is the crash window between minting a token and writing the anchor back — and the bit inside the card cannot be corrected without invalidating its fingerprint. So the number can never be quietly fixed. It is displayed, permanently, derived from on-chain evidence rather than from what the card asserts about itself. Most dashboards would have deleted this tile. It is the most useful one on the page.

Everything, decoded, in one place

Today's bloom, the KPI grid, rarity against the seeded roll weights, and the luck report.



CARDS DECODED

3

from 7 lakehouse rows read

ANCHORED ON CHAIN

100 %

of the collection, verified against Polygon

$SMILE MINTED

1,484

projected, including pending credit

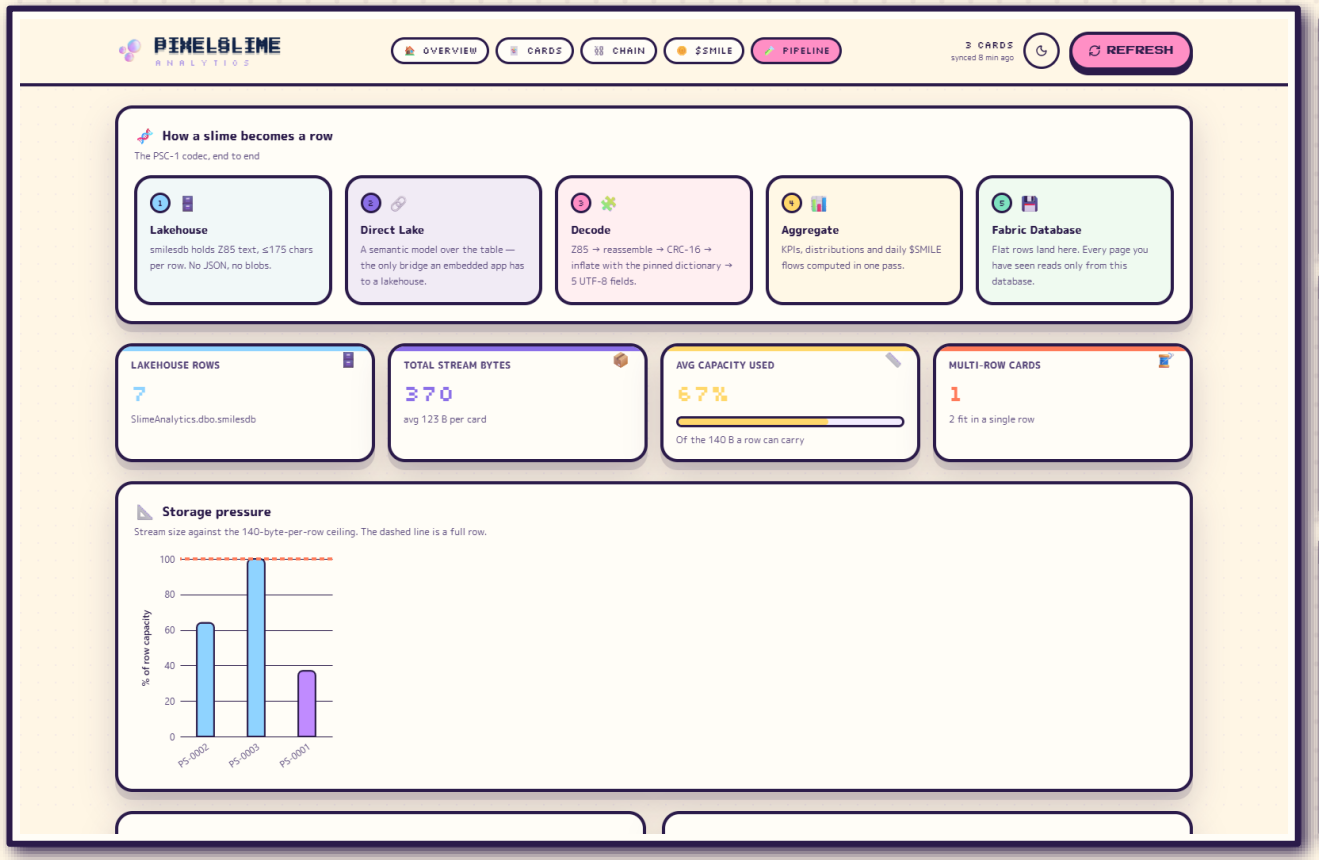
DECODE ERRORS

0

and chain drift is 0 as well

How a slime becomes a row

The one page in the app whose entire job is to explain the constraint that shaped the product.



LAKEHOUSE ROWS

7

raw rows read for 3 decoded cards

PIECE SIZE

140 B

binary payload per lakehouse row

THE WALL

175

characters, per row, non-negotiable

DICTIONARY

3,215 B

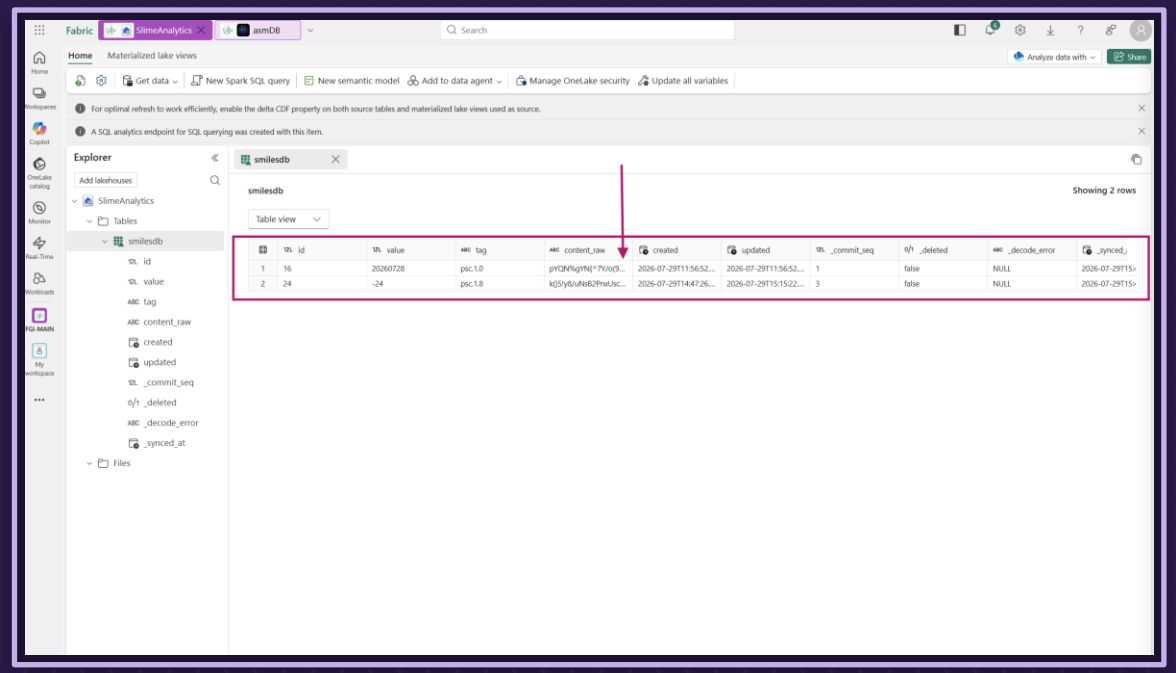
pinned, hash-checked on every decode

Storage pressure, as a first-class view

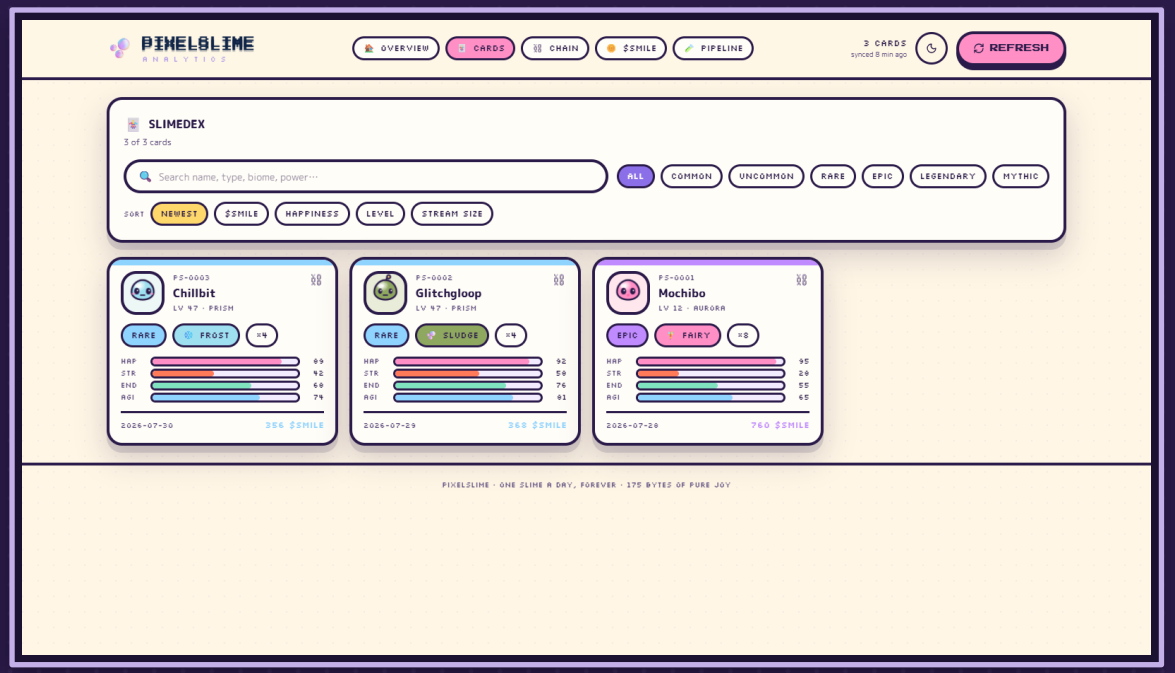
The page shows the largest card in the collection against the 175-character ceiling. When a future slime has a longer name and a wordier power, the chart is where that shows up first — before an insert fails in production at ten in the morning.

The row, and the row's reflection

Left: the bytes as asmdb stores them. Right: the same bytes, decoded by a Fabric app, four systems later.



THE LAKEHOUSE TABLE · content_raw, verbatim



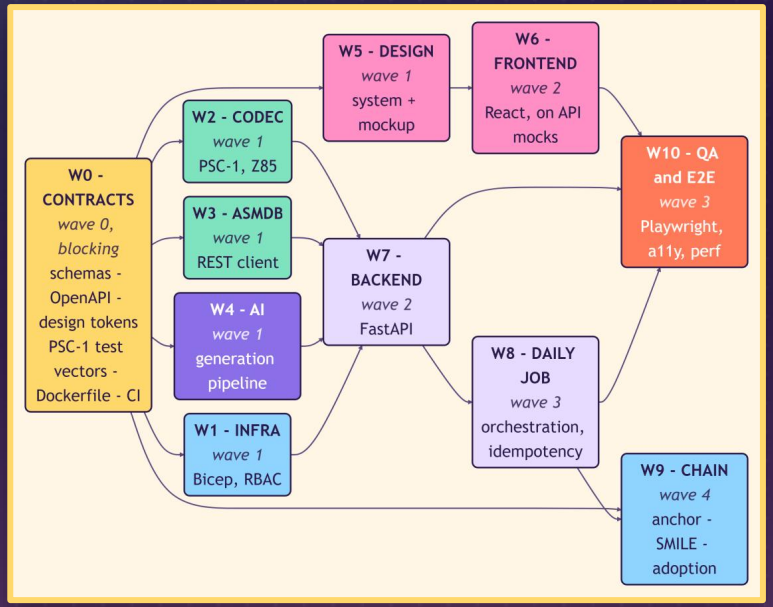
PIXELSLIME ANALYTICS · the same card, decoded

Nothing between those two pictures was hand-carried. An assembly engine wrote the row, a Go gateway published the change frame, a generated PySpark notebook merged it into Delta, a Direct Lake semantic model transported it, and a TypeScript decoder in a browser turned it back into a slime.

06

THE METHOD

How this was actually built



Five systems, one person, and a lot of agents. The interesting part is not that AI wrote code — it is what had to be true before letting it.

Agents do not need supervision. They need boundaries.

Parallel agents fail for exactly the same reasons parallel teams fail: unclear ownership, unstable interfaces and a definition of done that lives in someone's head.

WHAT GOES WRONG

Two agents, one file
They will both be right, and the second one will silently undo the first.

"Just make it work"
Produces code that passes no test, because no test was named.

An interface invented mid-task
Every other agent is now building against a contract that no longer exists.

A mock that was convenient
Ships. Nobody notices until the demo.

WHAT FIXES IT

One directory, one owner
Ownership is a path, not a conversation. Conflicts become impossible rather than rare.

A written definition of done
ruff, mypy --strict, pytest, tsc, lint. Not opinions — exit codes.

Contracts frozen before code
Schemas, OpenAPI, design tokens and test vectors land first, and then stop moving.

Fake neighbours, never fake data
You may stub the agent next door. You may not stub the thing you are being judged on.

THE ROSTER

Eleven workstreams, each owning exactly one directory

This is the entire coordination mechanism for PixelSlime. There is no standup, because there is nothing to coordinate.

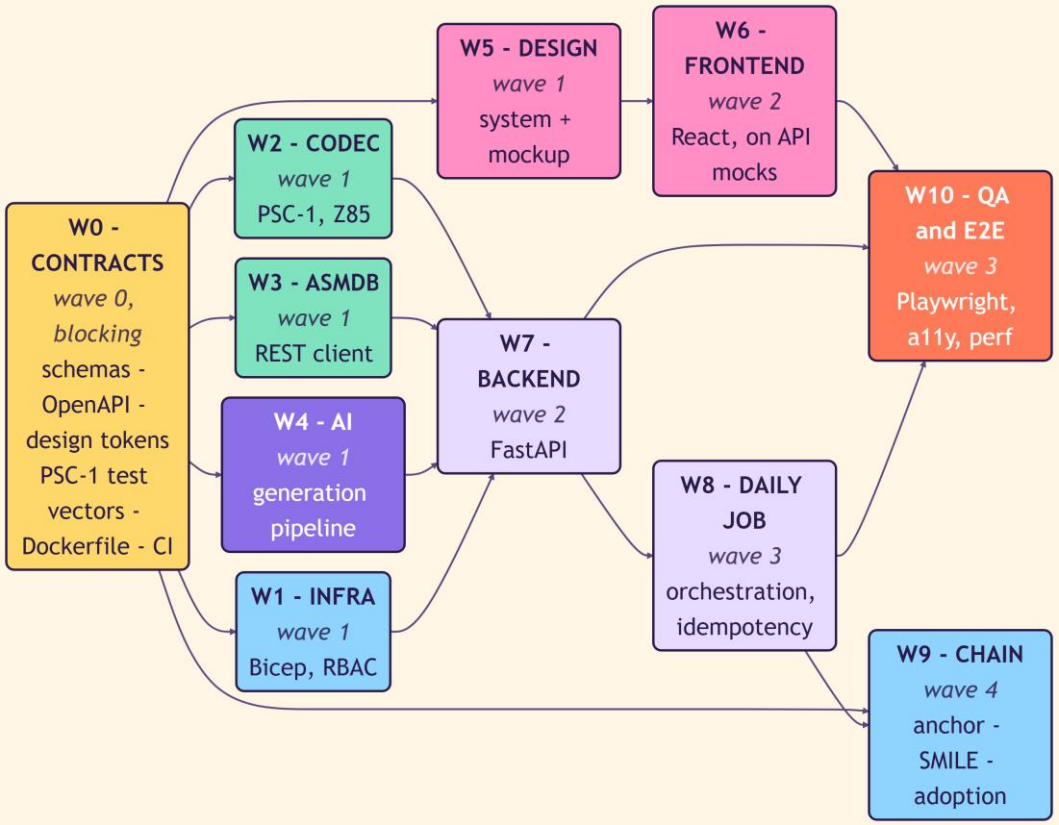
W0 CONTRACTS contracts/ · docs/ · .github/	**W1** INFRA infra/	**W2** CODEC backend/app/codec/
W3 ASMDB backend/app/asmdb/	**W4** AI backend/app/ai/	**W5** DESIGN frontend/src/design/
W6 FRONTEND frontend/src/ (except design/)	**W7** BACKEND backend/app/api/ · core/ · storage/	**W8** DAILY JOB backend/app/jobs/
W9 CHAIN chain/ · backend/app/chain/	**W10** QA & E2E tests/e2e/	

“One workstream owns one directory. Nobody writes outside their own.”

Eleven words. They removed the need for every merge-conflict conversation on the project, and they are enforceable by looking at a diff — which means an agent can check whether it is about to break the rule before it does.

The dependency graph is the schedule

Merge order is not negotiated. It is read off the graph: W0 → W1/W2/W3/W4/W5 → W6/W7 → W8 → W10 → W9.



WAVE 0 — **W0 alone**

Contracts and skeleton. Blocking, and deliberately short.

WAVE 1 — **W1 · W2 · W3 · W4 · W5**

Five agents in parallel. None of them can see each other's code, and none of them need to.

WAVE 2 — **W6 · W7**

The frontend builds against API mocks generated from the frozen OpenAPI; the backend builds against the real codec.

WAVE 3 — **W8 · W10**

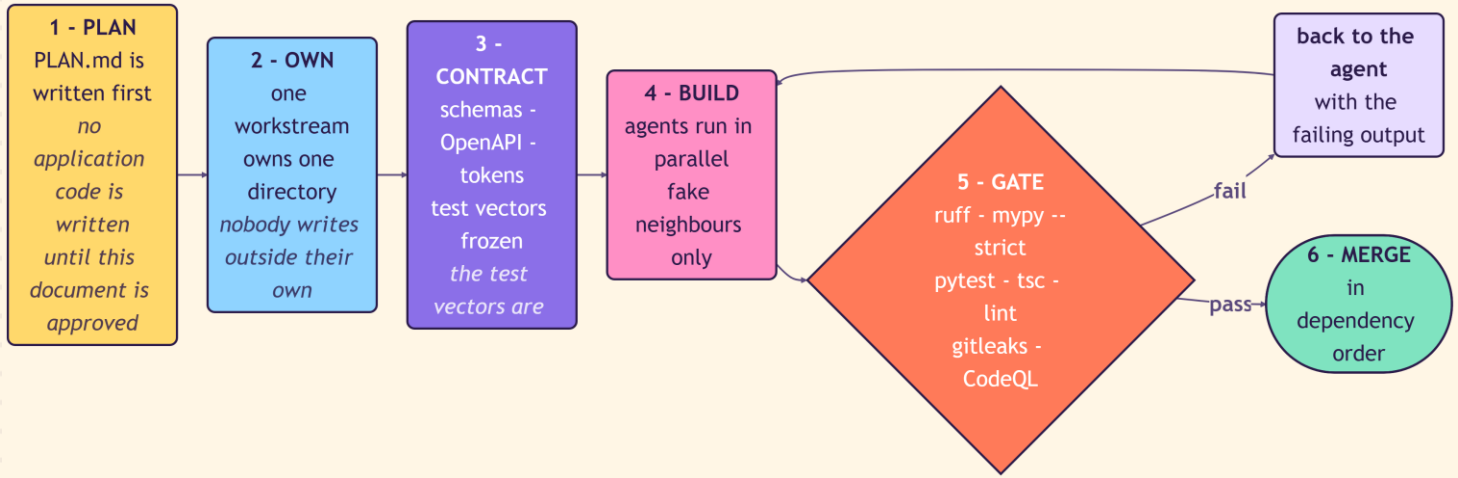
Orchestration and end-to-end. The first point where anything is judged as a whole.

WAVE 4 — **W9**

The chain last, because it is the only irreversible thing on the list.

Plan, own, contract, build, gate, merge

The loop only has one interesting edge: what happens when the gate says no.



The plan is a document, not a ticket

"No application code is written until this document is approved." The plan file is long, argumentative, and reviewed by a human before a single agent starts.

A failing gate goes back with its output

Not "it does not work". The exact ruff error, the exact mypy line, the exact failing test name. An agent given a stack trace fixes things; an agent given a complaint negotiates.

Merging is mechanical

Because ownership was exclusive and contracts were frozen, integration is a sequence, not an event. Nothing has to be reconciled, because nothing overlapped.

The test vectors are the law

W0 produces nothing that runs. It produces the things everyone else will be measured against — and then it gets out of the way.

JSON schemas

The shape of every object that crosses a boundary.

OpenAPI

The public API, complete, before any endpoint exists. The frontend mocks are generated from it.

Design tokens

Colours, spacing and type as data, shared by the mockup, the app and the poster.

PSC-1 test vectors

Known cards with their known byte streams. The codec is not correct when it looks correct — it is correct when it reproduces these.

Dockerfile and CI

The gates exist before the code they will judge.

“The test vectors are the law.”

Four words that make a codec agent unarguable with. It is not asked to write a good encoder — it is asked to reproduce these bytes.

Deliberately short

W0 blocks all ten other workstreams, so every extra day in wave zero costs ten agent-days. The plan says so out loud, which is why the contracts are minimal rather than complete.

THE GATES

Definition of done, as exit codes

An agent cannot argue with a non-zero exit status, and neither can I.

CONTRACTS

```
schema validation
requirements.txt sync
```

BACKEND

```
ruff check .
ruff format --check .
mypy app
pytest --cov=app
```

FRONTEND

```
npx tsc --noEmit
npm run lint
npm run build
```

IMAGE

```
docker build
```

SECRETS

```
gitleaks scan
```

SECURITY

```
CodeQL, weekly, JS/TS
```

Plus the rules that no linter can check: docstrings on every public function · no secrets in code or logs · no silent truncation or defaulting · docs updated in the same change · and, for the analytics app, **never mock, fake or hardcode data** and **never guess a schema — query it**.

A DIFFERENT SHAPE

PixelSlime Analytics had no roster — it had phases

A five-page app does not need eleven owners. It needs to stop guessing.

1 AUTHORING

Discover the real data and validate the queries first. Query exploration is delegated to sub-agents running in parallel, one per question.

2 DESIGN

Settle the UX before any runtime code exists, against real values pulled from the semantic model — never against invented sample rows.

3 APP CODE

Only now. And every UI change is followed by browser validation inside the Fabric portal, because that is the only place the app truly runs.

“Break tasks into independent pieces and delegate them to sub-agents running in parallel.”

The instruction is in the repository, not in a prompt I retype. Alongside it live custom skills for app validation, DAX authoring and query design — so “how we do this here” is version-controlled, reviewed, and applies to every agent that touches the project, including the one writing this deck.

TOOLING

What was actually wired in

None of this is exotic. All of it is checked into the repository, which is the entire point.

AGENTS & PROMPTS

- AGENTS.md per repository
- .github/copilot-instructions.md
- .agents/skills/ — app-validation, dax-authoring, query-design
- Sub-agent delegation, in parallel

QUALITY

- ruff · mypy --strict · pytest
- TypeScript tsc --noEmit · ESLint
- Playwright, including reduced-motion runs
- gitleaks · CodeQL

PLATFORM

- Bicep for every Azure resource
- Container Apps and Container Apps Jobs
- Fabric Workload Development Kit
- Rayfin CLI · fabric-app-data

MODEL SURFACE

- MCP — asmdb exposed as an agent tool
- gpt-5.6-sol for structured metadata
- gpt-image-2 for artwork
- Foundry and web3.py for the chain

MCP matters more than it looks: because asmdb speaks it, an agent can create a database, insert rows and verify durability against the real engine — not against a description of it.

RETROSPECTIVE

What worked, and what I would do differently

The failures are more informative than the wins.

WHAT WORKED

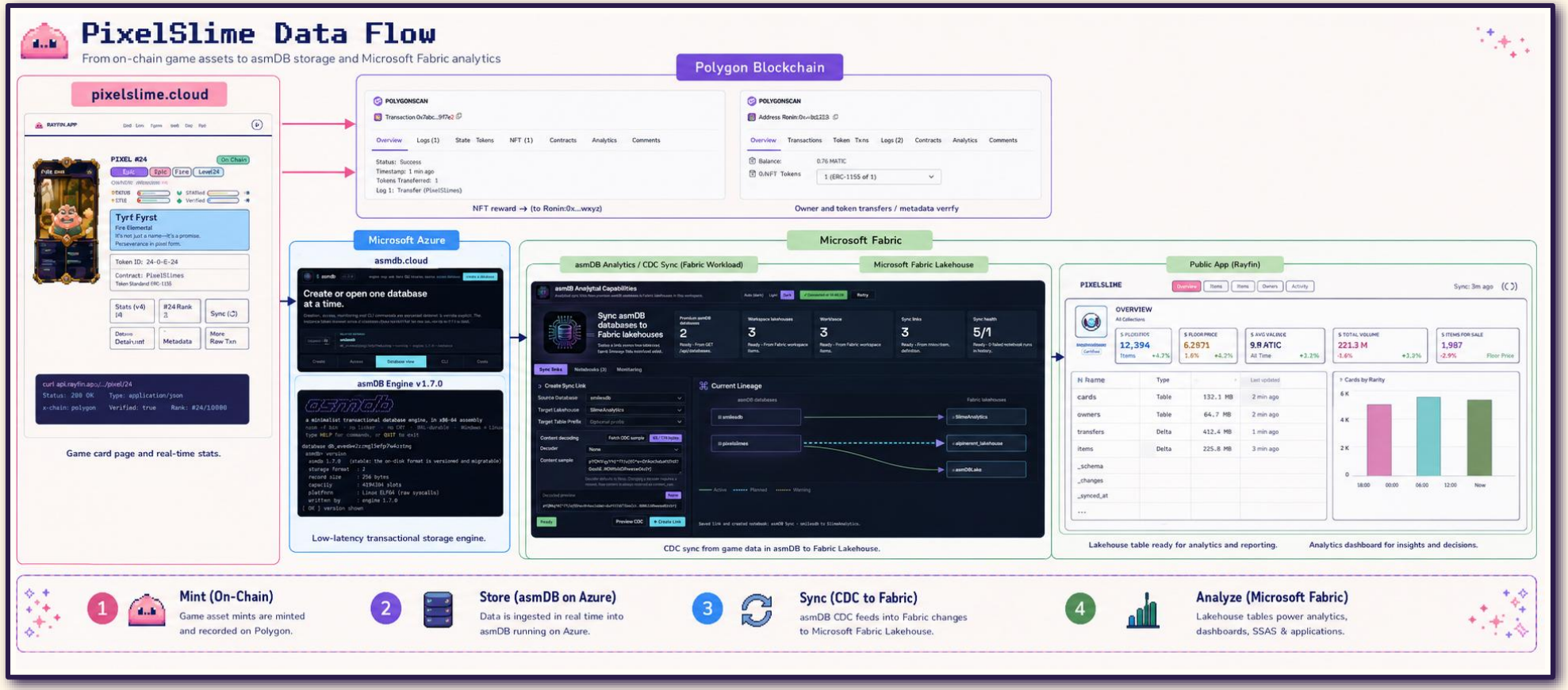
- Exclusive directory ownership. Zero merge conflicts across eleven workstreams.
- Freezing the codec test vectors first. The encoder and the decoder were written by different agents and matched on the first integration.
- Generating frontend mocks from the OpenAPI file, so the UI was built before the API existed and still fitted it.
- Making CI the definition of done. “Is it finished?” became a question with an exit code.

WHAT I WOULD CHANGE

- Wave 0 was still too long. Every day in contracts costs ten agent-days downstream, and I under-priced that.
- “Never mock data” had to be written down twice, in capitals, before it stuck. Convenience mocks are the single most common failure mode.
- Browser validation should have been a gate from day one, not a rule. Rules get skipped; gates do not.
- The chain went last, which was right — but it meant the anchor crash window was discovered late, and it is now a permanent metric rather than a fixed bug.

The whole thing, on one page

From an on-chain game asset to asmdb storage to Microsoft Fabric analytics — mint, store, sync, analyze.



THE NUMBERS

Everything that matters, in one place

ENGINE

Binary, Windows
43,741 B

Binary, Linux
52,205 B

Record size
256 B

Usable rows
3,145,728

In-RAM insert
12.38 M rows/s

Rows per transaction
4,096

SERVICE

Tiers
$0 / $15 / $49

Platform floor
$141.76 / mo

Replicas per instance
1

Public endpoints
1

NFS share
100 GiB

Gateway timeout
60 s

PRODUCT

Bytes per card
175

PSC-1 header
32 B

Mochibo stream
52 B / 65 chars

Genesis Rain
365,000 $SMILE

Burn per bloom
100 $SMILE

Total blooms
3,650 = 10 years

PROOF

Backend tests
726

Chain tests
49

MCP checks
24

Contracts on chain
4

Decode errors
0

Chain drift
0

Constraints are not obstacles you route around

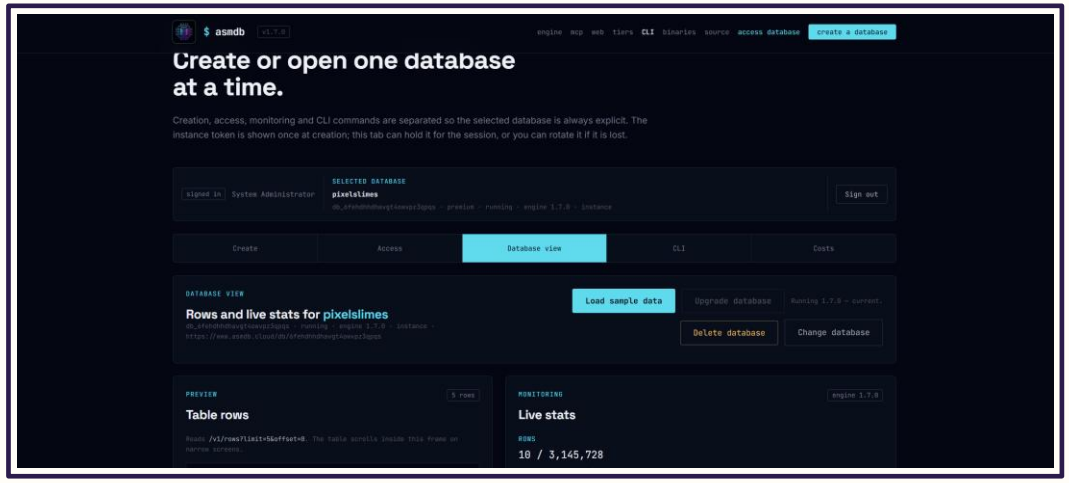
Every layer in this stack got better when it was told what it was not allowed to do.

256-byte records	→	made the address arithmetic two instructions and deleted the entire storage manager
No SQL	→	removed the parser, the planner and every class of injection that comes with them
One writer	→	made durability provable instead of probabilistic, and made isolation a container boundary
175 bytes of content	→	forced PSC-1, which is the most interesting engineering in the product
One database per Container App	→	made the blast radius exactly one customer, and made upgrade an image tag
One directory per workstream	→	made eleven parallel agents cost nothing to coordinate

GO AND LOOK

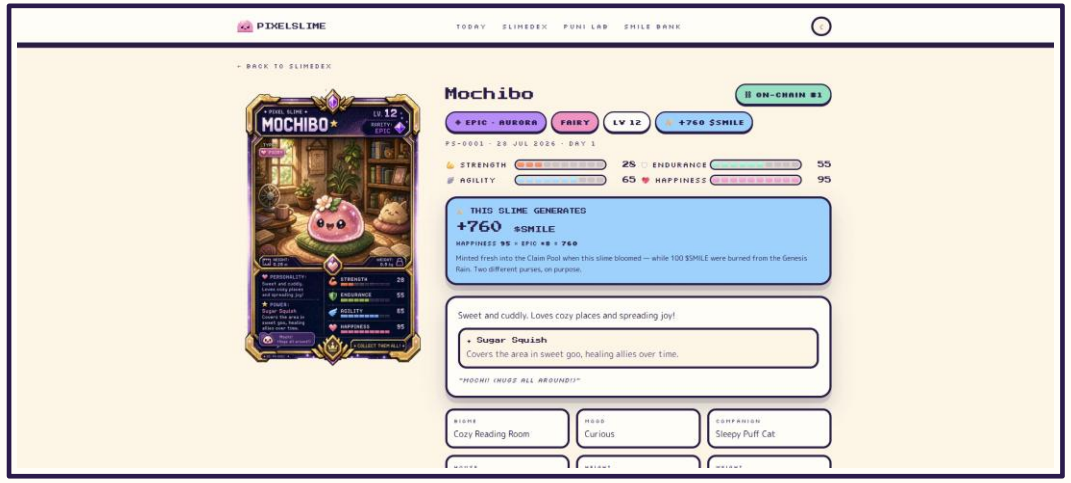
It is all live, and it is all checkable

None of this needs an account, an NDA or a demo environment.



www.asmdb.cloud

The engine, the console, the CLI, the tiers and the costs page. Create a free database and run VERSION.



www.pixelslime.cloud

Today's bloom, the SLIMEDEX, and the UNSEAL panel that shows you the exact bytes behind every card.



One slime a day, forever.
175 bytes of pure joy.

Thank you. Questions, arguments and “why didn't you just use SQLite” all welcome.

WWW.ASMDB.CLOUD

WWW.PIXELSLIME.CLOUD

ASMDB.ANALYTICS

PIXELSLIME ANALYTICS